

软件仓库管理 (2) [04-Raw/26生成式软件工程/NJU]

原视频: <https://www.bilibili.com/video/BV1Q6en6NEUo> · 课程合集:
<https://space.bilibili.com/202224425/lists/8942017> · 讲者: 蒋炎岩 (NJU 《生成式软件工程》2026)

全课主线

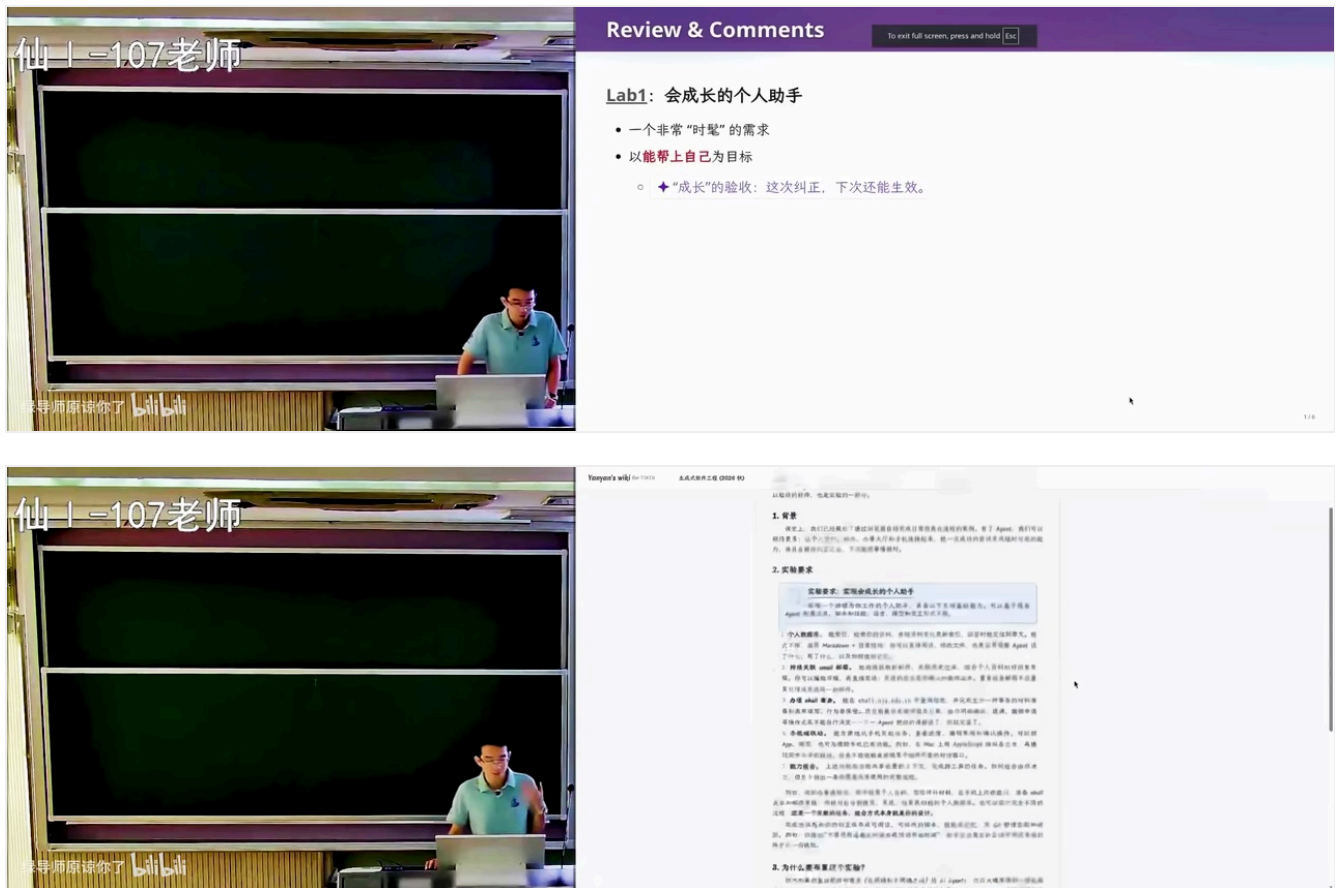
讲者想让听众相信: agent 时代不能把任务直接丢给 AI, 也不能继续照搬只为人类设计的 Git/版本管理; 要回到 intent、spec、架构、MVP 和团队管理, 并用经典 CS 概念重新设计面向 agent 的版本管理系统。学生现在就该边用 agent 边建立项目管理和版本控制直觉, 先小步、保留快照、管理 agent team, 而不是无脑 vibe coding。

1. 讲者先以本周继续版本管理、作业验收和 Lab1“会成长的个人助手”切入, 鼓励 vibe coding, 并构想 always on 的个人助手。(第 1 部分)
2. 但是, 如果直接把实验要求交给 AI, 它会一开始做很多看似合理的决定, 项目越来越难维护直到焦油坑; 因此实验要让大家经历这个坑, 下一 Lab 还会加冲突需求让你越改越崩溃。(第 2 部分)
3. 于是重点回到项目本身: 项目是 intent、设计文档、README 等文件的集合, README 应短到能表达原始 intent、说服合作者, 内容链条是 intent→spec→code implementation。(第 2、3 部分)
4. 然而文件系统只是历史遗留的方便存储, 会丢失 intent/spec、测试、代码之间的隐藏关联; 文档与代码各自演进、不一致累积会形成 technical debt, 进入维护泥潭, 这就是 traceability 问题。(第 3、4 部分)
5. 因此讲者设想未来软件是对象/实体之间有明确关系、甚至有环的图, 放进数据库而不再是文件树; 但人类用现有 UNIX 工具维护不了含 cycle 的 repo, 所以要用 AI (有强化学习反馈) 理解 repo、自动维护 artifacts 一致性, 仍按 Git 式小步 commit, 并由 AI 从 chat 抽取相关部分动态生成 view。讲者强调这靠数据库 view、实体关系、Git 小步等经典 CS 概念, 所以学生不应因新范式放弃经典课程。(第 4、5、6、7 部分)

6. 回到 Git 本身：Git 管快照并要求小步 commit，但两路 merge 会把 B 的改动/冲突解决记到 A 头上；rebase 逐个搬 feature/test 追求线性历史，代价是冲突和 commit ID 重写。于是讲者指出 Git 没有专门 change 对象，只用 commit+parent 表示修改；若把 change 做成一等公民，就能得到下一代 VCS：rebase 可搬运 change，冲突可随 change 带下去、稍后 resolve，并且没有 staging area。（第 7–11 部分）
7. 但旧 VCS 都默认使用者是人类：人类单线程、读写不能同时、沟通慢，commit 以小时/十分钟计，所以 stash、merge/rebase 等为人设计是合理的；agent 可以秒级产生原子提交、轻易形成 10 倍/100 倍 team，因此必须重新设计面向 agent 的版本管理。（第 12、13 部分）
8. 在初期，讲者主张让 agent 沿用人的模型：— agent —开发机/容器/远程仓库，从 tmux 接命令，监工 agent+子 agent；同仓库有冲突，git worktree 是轻量隔离 workaround，但冲突仍需 merge/rebase。（第 14、15 部分）
9. 进一步，人的沟通成本随人数平方上升，团队不能线性放大；Git blame/bisect 服务于人类追责，agent team 更关注修错和前进，但快照仍必须保留。讲者又把 Git 类比乐观并发控制：agent 变快后会频繁冲突/abort，于是可能转向按 scope 上锁的悲观并发控制，代价是互斥锁排斥并行。（第 16–19 部分）
10. 所以讲者判断版本控制只是工具，真正难的是进度和团队管理；只要 AI 还不能一句话做出微信级产品、也还不会团队管理，软件工程就仍需要，最稀缺的是能分解任务的 team leader。（第 19、20 部分）
11. （整理者串联）随后讲者转到产品与信息系统：AI 助教/playground 等例子显示 AI 会过度遵循 prompt，完成字面指令却偏离真实 intent；自动 git commit 被他认为比过早 persist 到数据库更贴近版本管理的本质。（第 21–24 部分）
12. 因此他主张项目早期不要 multi-agent，先当包工头用最强 agent 多轮对话；第一件事是建空仓库，再小步走，并做 MVP，聚焦绝对要的学习/工作效率需求，可用熟悉的文件系统和云盘，数据库等性能瓶颈再迁移。最后落回架构：架构要以最小变化应对需求变化，版本管理要记住现在、历史和教训，早期 prompt 影响最大。（第 23、25、26、27、28 部分）

讲者的结论/号召是：现在就开始用 agent 做项目，但要像管理团队一样管理它，先小步、MVP、保留快照、练 team leader 能力，并继续学经典 CS 以重新设计 agent 时代的版本管理与软件工程。

第 1 部分



本周继续版本管理：作业自愿，但最终要验收并由机器人评分

- 讲者说：上周已经讲了一点**版本管理**，这周继续讲版本管理。
- 已经发布作业。按之前的说法，这门课所有作业都是**选做、自愿完成**。
- 但讲者补充：最后当然会**验收**。原因是学校最终也许会让提交一个成绩。
- 讲者给出的流程：作业自愿 → 最后仍要验收 → 学校可能要成绩 → 大家交一些链接 → 由讲者的**机器人**评分。

第一个 Lab 是“会成长的个人助手”：做法不限，鼓励 vibe coding

- 第一个 lab 被讲者称为一个比较时髦的需求：会成长的个人助手。
- 对大家的要求是**不限**，想怎么做就怎么做；最好还是可以 **vibe coding** 一个（原记录作“web code”）。
- 今天课上应该会留时间，讲者自己会现场 **vibe coding** 一个。

- 他会告诉大家：如果直接让它 vibe coding，会遇到什么样的小坑。
- 逻辑链：Lab 需求不限 → 鼓励 vibe coding → 讲者现场做 → 提醒直接 vibe coding 的小坑。

课程安排：这周日补周二课，届时讲软件架构并串起内容

- 讲者纠正时间安排：不是下一周，而是这周日要调休、补周二的课，所以这周日还有课。
- 到时会讲**软件架构**，并且会把它“连起来”。
- 随后讲者给出例子：每个同学都会需要、他自己也有的，是一个能够持续为你工作的**个人助手**。
- （整理者串联）本段把后续软件架构课与**个人助手 Lab** 放在同一条连续线索上；具体如何连接，讲者这里没有展开。

个人助手不是抽象作业：学习、科研、填表都能落地

- 讲者说这些需求只是**供参考**；如果实现它，会有一点**成就感**。
- 学习场景：
 - 可以把专业课资料放进个人数据库。例如本学期可能在上**计算机网络、数分、概率**之类比较难的专业课。
 - 参与研究组课题时，可以把研究组论文、相关文献、教科书整理成个人数据库。
- 数据库形式不限。
- 填表场景：
 - 讲者观察到填表很烦：每次给一个 DOC 表格，要填姓名、学号、身份证号，有时还有个人简历等。
 - 讲者现在已把这些事情交给 **agent** 填。
 - 讲者认为从以往到今年有非常大的**效率进步**。

从自动填表系统到 ChatGPT：讲者用亲历说明变化之快

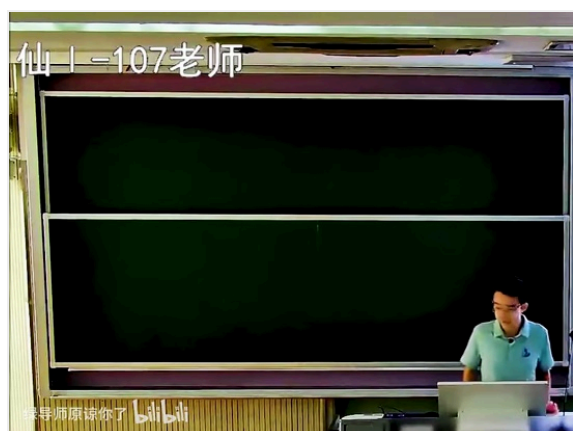
- 讲者回忆：大概在疫情时招过一个学生，研究生在他这里做了 3 年，做的就是**自动填表系统**。
- 学生毕业时突然有了 **ChatGPT**，他很慌，觉得自己做的事情马上要被替代。

- 讲者对他说：不要紧，赶快答辩，然后就可以毕业。
- 讲者借此表达：世界发展就是这么快。

从个人数据库到 always on：关联邮箱，在宿舍跑 agent loop

- 在个人数据库的基础上，可以把数据库和你的 **SMail 邮箱** 关联起来。
- 讲者举例：他可能两周后会在半夜给大家发邮件，你的 **agent** 可能会把邮件回给他，实现一个 **always on**、在后台工作的助手。
- 设备方案：也许买一个很小的**嵌入式小设备**，一两百块钱，就可以在宿舍里持续跑你的 **agent loop**。
- 最后一步构想：用你的信息可以办理**一般的事务**；讲者说，就像他上课时说的，这就是一句话，直接一个像 agent 的**指令调用**就可以。
- 逻辑链：个人数据库 → 关联邮箱 → agent 在后台处理邮件 → always on → 小设备持续跑 agent loop → 用信息办理一般事务。

第 2 部分





Yanyan's wiki (for TOKEN)

生成式软件工程 (2026 秋)

3. 为什么要布置这个实验？

因为如果你直接把这些需求（在明确和不明确之间）给 *AI Agents*，你将大概率得到一团乱麻的 *AI slop*——它也许的确可以工作，但每一次修改都会消耗大量的 *tokens*，*Agents* 会在泥潭中反复阅读代码、试错，像极了《人月神话》中的软件集油坑。从而迫使你思考应该如何重新组织项目——但这是好事，因为 *AI* 重构的能力极强，一旦你找到一个更好的架构，瞬间就会改进。

请留存好你的项目，它可能会是你成为 *AI-native* 新人类的起点——之后我们也会持续地改进它。



Yanyan's wiki (for TOKEN)

生成式软件工程 (2026 秋)

3. 为什么要布置这个实验？

因为如果你直接把这些需求（在明确和不明确之间）给 *AI Agents*，你将大概率得到一团乱麻的 *AI slop*——它也许的确可以工作，但每一次修改都会消耗大量的 *tokens*，*Agents* 会在泥潭中反复阅读代码、试错，像极了《人月神话》中的软件集油坑。从而迫使你思考应该如何重新组织项目——但这是好事，因为 *AI* 重构的能力极强，一旦你找到一个更好的架构，瞬间就会改进。

请留存好你的项目，它可能会是你成为 *AI-native* 新人类的起点——之后我们也会持续地改进它。



Yanyan's wiki (for TOKEN)

生成式软件工程 (2026 秋)

Creative Commons License: BY-NC 4.0



Review & Comments

Lab1: 会成长的个人助手

- 一个非常“时髦”的需求
- 以能帮上自己为目标
 - ✦“成长”的验收：这次纠正，下次还能生效。

让 Agent 自己完成任务：一句话调用，甚至可做手机端联动

- 讲者说，让 Agent 用你的信息去办理某些事务，可能就像上课讲过的那样，只是一句话；直接让 Agent 调用/执行就可以。
- 如果你们有兴趣，还可以把 Lab 做成**手机端联动**：网页、iOS app、Android app 都行。
- 讲者认为，只要让 Agent 去做，它会自己帮你下好所有需要的东西，然后把你要的东西做出来。

任务都在 AI 能力范围内，但直接让 AI 做会让项目难维护

- 表面上看，所有任务都在 AI 的能力范围之内。
- 但讲者提出：如果直接把实验要求交给 AI 去做（他说自己没试过直接这样做），AI 一开始就会做出很多它认为“看起来合理”的决定。
- 这些决定会让项目维护变得越来越困难，最后直到陷入软件工程里的**焦油坑**。
- 讲者认为每个同学都有必要体验一下这个焦油坑。逻辑链是：
 1. AI 一开始自主做很多看似合理的决定。
 2. 这些决定持续积累，项目越来越难维护。
 3. 当你的 AI agent 开始不断消耗你的 token 时，你总有一天会下决心：把项目删掉、从头重做，或者回退到某个很早期的版本再重做。
 4. 这就是这个实验想让大家经历的事情。
- 讲者相信这个项目对每个同学都是有用的，这也是他设计实验的初衷。

下一个 Lab 会故意制造“越改越崩溃”

- 讲者预告：下一个 lab 会和这个 lab 有非常大的关联，而且你们暂时猜不到。
- 情景是：lab1 的实验要求都做好后，它会成为一个非常舒适的个人助手；然后加一个需求，这个需求与前面所有内容都关联，并且有一定冲突。
- 你只能去改，然后越改越崩溃。讲者希望大家期待后面的 lab。
- 反过来，如果 lab1 选择躺平、不管它，直接让 AI 做，下一次可能就把做的直接删掉；讲者认为这样可能就没有软件工程的体验了。

回顾项目管理：项目是 intent、设计文档、README 等文件的集合

- 上节课讲的是软件项目管理。项目是各种文件的集合：你的 **intent**、一些设计文档、你的 **README** 都在项目里。
- 讲者不喜欢 AI 写出来的非常非常长的 **README**。
- 讲者喜欢的 **README** 是：写你在设计这个项目时最初的初衷——你想做一个什么样的产品，这个产品能达到什么样的效果；可能放两个 **screenshots**；以及你最重要的那些设计。

README 的用途：短到能说服别人加入你的“挣一个亿”计划

- 讲者的类比：有一天你突然想到一个非常好的点子，真的想用它去挣一个亿。
- 想挣一个亿时，你要组建一个创业团队，要说服 computer science 里的人，比如说服讲者出来干。
- 这时你其实只需要一个比较短的 **README**。
- 讲者认为，这就是你的 **intent**：你想做这个东西。

第 3 部分

仙上=107老师



导师原班你了 bilibili

项目是各种文件的集合

Intent、Spec、Code、...

会成长的个人助手

 intent.md 想解决什么	 spec.md 约定行为	 assistant.py 实现
 test_assistant.py 测试	 memory.md 记忆	 README.md 使用说明

2 / 6

说服 CS 合作者：只需最短 README 表达原始 intent

- 讲者设想：你要说服一个 **computer science** 背景的人出来一起干。
- 对方其实只需要给一个较短的 **README**，写清 **intent**：你到底想做什么。

- 剩下的比如“我有架构能力，我有写代码能力，我可以把剩下的东西给你补出来”。
- 因此，提出 intent 的人不需要先知道完整软件系统：**specification** 是什么、用什么数据库、怎么支持用户从 1 到 10、100、100 万、1 亿。
- 讲者强调：这个阶段你可能没有 **computer science** 的知识，但你不需要有；你只需要有最原始的设计，也就是 **intent**。

项目内容链条：intent → spec → code implementation

- 讲者给出的层级是：
 1. **intent**：最原始的设计，想做的东西。
 2. **spec**：比如选什么数据库、选什么编程语言、项目结构是什么；小到每个模块的接口都属于 spec。
 3. **code implementation**：代码实现。
- 这些内容现在都放在项目里。
- 有这样一个项目以后，很自然就需要一个 **容器**，把这些各种各样的文件管理起来。

文件系统管理项目：图方便 + 历史遗留

- 大家用 **文件系统** 来管理项目，讲者认为其实真的就是图方便。
- 历史原因：
 - 回到 1950、1960 年代，操作系统给你的存储功能就只有文件，没有别的。
 - 那时关系型数据库还没有发明出来。
 - 所以所有程序员别无他法，只能把项目以文件形式存进来。
- 因此这更多是历史原因，历史不停往后涨、往后涨导致的。
- 讲者认为：**文件系统可能不是组织项目的正确方式**。

文件系统丢失了“不适合由文件表达”的信息

- 文件系统丢失了很多“不适合由文件表达”的信息。
- 有些信息本来比较容易表达：
 - **intent/spec**：一条文档，写“这个地方我是怎么想的”。
 - **测试用例**：输入是什么、输出是什么，以及在程序运行过程中 **assert** 什么东西不能被违反。

- 但很快会发现：项目里有很多 **隐藏关联** 没有表达出来。

例子：linked list 文档与 tree 代码之间的隐藏关联

- 讲者举了 AI 写代码的例子：
 - 现在 **agent** 会稍微好一点，有时也不会。
 - 你让 AI 把某个实现方法从 **linked list** 换成更高效的 **tree** 结构来优化性能。这很常见：突然发现性能不够，就让 AI 去做。
 - AI 会啪啪啪把代码改掉。
- 但你可能没想到：某个文件里的 **markdown 文档** 写着：“我这个项目的数据是用 **linked list** 管理的。”
- 于是文档、代码、测试之间形成了一个关联。
- 问题在于：写文档时可能还没有代码，所以先随手写了“这是个 linked list”。等代码实现以后，这个关联又没有强行联系上。
- 这就成为 **technical debt**（技术债）。

结果：不一致只会越来越多，软件陷入维护泥潭

- 讲者给出的因果链：`文档随手写 linked list → 代码改成 tree → 文档/代码/测试的隐藏关联未被强制联系 → technical debt → 文档与代码不一致增多 → 维护泥潭`
- 因为以后随着文档和代码不断向前推演，它们的不一致只会越来越多，不会越来越少。
- 最终这个软件就陷入 **维护的泥潭**。

小结：讲者主张文件系统是历史惯性而非组织项目的正确方式；隐藏关联不被强制表达，会积累成 technical debt。

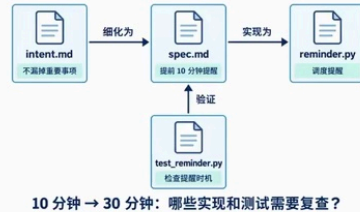
第 4 部分



文件装得下什么？

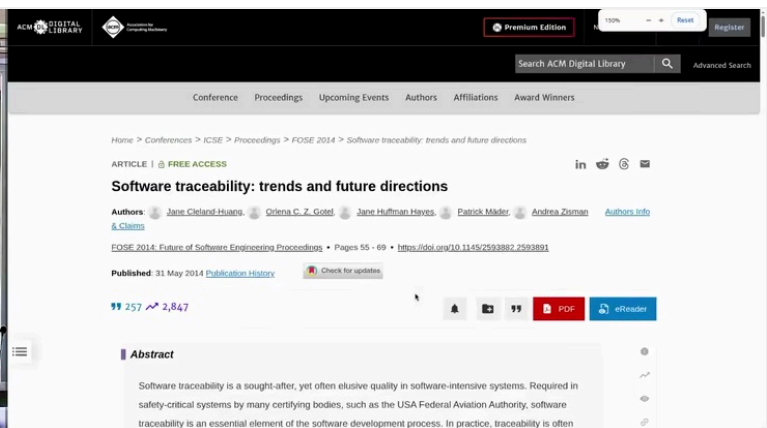
文件系统的方便

- 其实“文件”只是文件系统的方便
 - 其他形式 (例如 db) 对人类和工具都不太友好
- 但丢失了很多文件不适合表达的信息。例如 artifact 之间的 *traceability*



10 分钟 → 30 分钟: 哪些实现和测试需要复查?

3/6



文档与代码各自演进 → 不一致累积成 technical debt → 维护泥潭

- 讲者核心判断：随着文档和代码不断向前推演，二者的不一致只会越来越多，不会越来越少。
- 结果：软件陷入维护泥潭。
- 具体困境：后面你拿出一份代码，甚至 AI 看到一份代码，到底是文档说得对，还是代码说得对？
- 判定依据会丢：聊天历史没了，chat 消失；换了一台电脑；Git repo 里可能还有一些 change 日志，但到最后真的不知道到底是代码对还是文档对。
- 讲者的推理链：
 1. 文档与代码继续演进；
 2. 二者不一致累积；
 3. 形成 **technical debt**；
 4. 软件进入维护泥潭；
 5. 代码/文档谁为权威变得不可判定。

- （整理者串联：第 5 步是第 4 步的具体表现；讲者用 AI 看代码、chat 历史消失、换电脑、只剩 Git change 日志来说明判定依据的丢失。）

这个问题在软件工程里叫 **traceability**（可追踪性）

- 讲者说，这是软件工程里研究了非常非常长时间的问题，叫 **traceability**。
- 他提到 2014 年已经有 review 在总结更老的文章，题域可写作 **software traceability**。
- 所谓追踪：一个软件项目里的各种测试用例、文档，哪怕文档中间的一行代码，它们之间的关联是什么；以及我们能不能在软件系统演进过程中仍然维持这种关联。
- 讲者认为这其实是一个非常软件工程的问题。

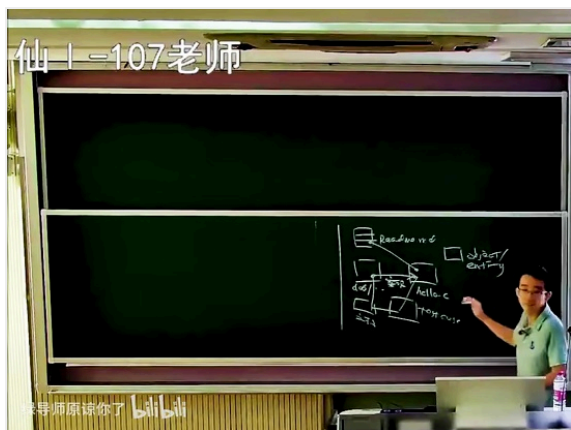
文件系统只是“将错就错”的方便存储；SVN/Git 之后讲者追问下一个

- 讲者提到：文件其实只是图，保存在文件系统里只是方便。
- 用文件系统管理代码，大家总有一天会受不了：打压缩包发布，打压缩包发给合作者，这件事已经维护不下去。
- 于是有了 **SVN**，又有了 **Git**。
- 讲者追问：下一个是什么？他预告后面会讲另一个版本控制工具（GGT[待后文确认]），并认为它可能甚至比 Git 更好用。

若未来人不再直接写代码，软件应变成对象/实体的关联图

- 讲者设想的未来：如果以后我们都不再写代码，人几乎不管这些东西，软件就不应该像现在这样在 Git repo 里以一堆文件来管理。
 - 现在的文件形态包括：README.md、hello.c、DOS 下的文档等。
 - 未来也许仍然需要文档，但文档可能是很多个切片，被变成很多 **object / entity**。
 - 这些对象之间有明确关系：
 - 代码实现某个文档；
 - 测试用例关联某个文档；
 - 测试用例关联代码的某部分实现；
 - 文档又和文档关联。
 - 讲者强调：这是一个完全结构化的东西，无法在文件系统里以树状形式表达；它是随便连接、有环的图（**graph**）。
 - 因此这个东西，比如可以放到一个数据库里。
-

第 5 部分



将错就错，然后呢？

基于文件的工具蓬勃发展

- 从压缩包到 SVN、Git、...
- 人类只能将错就错

AI 时代的软件形态

- 推论：AI 时代“软件”的形态会发生根本的变化



```
从白板-Agent-到完整生态-d78c820f299ee825a7e08c537bf5def13c4308f.md
使用-Agents-CS-Professional-视角-50e33d1ec5954b261a015e17e74fdb0154ef3430.md
使用-Agents-普通视角-68fad150b336017d7fbeb8e648df7a8fab70bed.md
古法编程-死了-19100ea8a0e897ba188adc7f7ce95f00fb577ba1c.md
我们站在一个重要的交叉路口-dce32ef8bc6dcee790cd212f02f0b9456ac89446.md
软件工程-18da43255203eca48a46d9f29a6f798b4095169d.md

p2
Long-Horizon-能力怎么练-练出来的-7604234f042bfeb727a981f6a8287fe412c8e3e1.md
一步很浅-一串可以很深-6fda6fe71664248c58053b071cd7f1086ce37f67.md
上下文工程-让模型知道-完成-是什么-ff1fac7a86f76a1ccc620d8de2cafea872fc7a1.md
在不确定性中探索-5b669555d371b02883d9689649a0ba591474b12.md
居然以一个非常意外的方式上热搜了-652ac60f83cfb6784e2bf82bd4b4e3c57b271d2b.md
提示词-从哪里来-cc4f28347387f44a421581c0e340537187605357.md
用好模型遵循指令的能力-36bea1d86d3ad17a09885a35bc877ac262a110e.md

p3
Anyway-我们还是要学习的-311c9600137af3045a5e07da75eafa04f79f3782.md
Git-听说过-还是用过-f0926e7b2f765fb18970744ee6a1b122325182e4.md
从空白开始-a0d0535894c0220aed62f5476738498c23a0bead.md
告别古法学习-00efee1c22e3f35e5016ebd0c38a64a00c32073.md
即刻软件工程-cb75026c2bec199067a4d81c423e99375f956a3a.md
快讯-GPT-6-发布了-ffefa58826a551dacdd312f41148c96817a64618.md
更多的-Best-Practices-a8c6e43ec1f768d8b2bd004090a989e1b5d474a.md

p4
Agent-时代-再次重新设计版本管理系统-649eefdbf3436c0fe50ff19ee12bc6300c606514.md
Review-Comments-11adcd2152b08f531d09ab3e79b952ba1e551aa1b.md
团队协作-4be09d378c940df7cfb6a1b2bbda508709550e24.md
如果想重新设计版本管理系统-92cbf5178f4c15f4203e0eb987b0bbc59e4795fb.md
开始做一个项目-a5b3a18d51e70cfb99a9f5a89704aad6dcbab60c.md
```

7 directories, 36 files

含 cycle 的图/仓库难以用 UNIX 工具维护，讲者要用 AI 来维护

讲者的逻辑链：含 cycle 的 graph/repo → 放进数据库 **repo.db**（原 ASR: report.db）→ 现有 UNIX 工具无法维护 → 训练 AI（有强化学习反馈信号）理解 repo → AI 自动维护 artifacts 一致性 → 修改仍按 Git 式小步 commit → 小步只需局部上下文 → AI 从 chat 中抽取相关文件/test/doc，忽略其余。

- 讲者先给出一个随便连接的 **graph**，其中有 **cycle**。
- 这种结构可以放进数据库，讲者把它叫 **repo.db**（原 ASR: report.db）。
- 讲者判断：人类没有办法用现在 **UNIX 世界里的工具**维护这样的 repository。
- 解决思路是“用魔法打败魔法”：人类理解不了的东西，就训练 AI。
- 关键前提：只要有强化学习反馈信号，就可以训练 AI，让 AI 能理解 **repo**（原 ASR: RO）。
- 讲者把这件事说成一个非常大的项目，例如 **Linux 内核数据库**，或者“一个价值一个亿的产品”。

软件已工作时，修改应像 Git 一样小步前进

- 讲者指出：不可能一下子把产品直接变成另一个产品；当软件系统已经在工作，一般就是给它增加功能、修改功能。
- 这与 Git 的 philosophy 很像：Git 管理文件/目录，以及快照和快照之间的关系。
- Git 式做法是小步前进：每做一件事、fix 一个 bug、改一个文档，都产生一个 commit。
- 这样可以在多人之间协作：每个人产生自己的 commit，再用 rebase、merge、fast-forward 或普通 merge，创建可追踪的整个项目修改历史。

核心目标：AI 自动维护 artifacts 之间的一致性

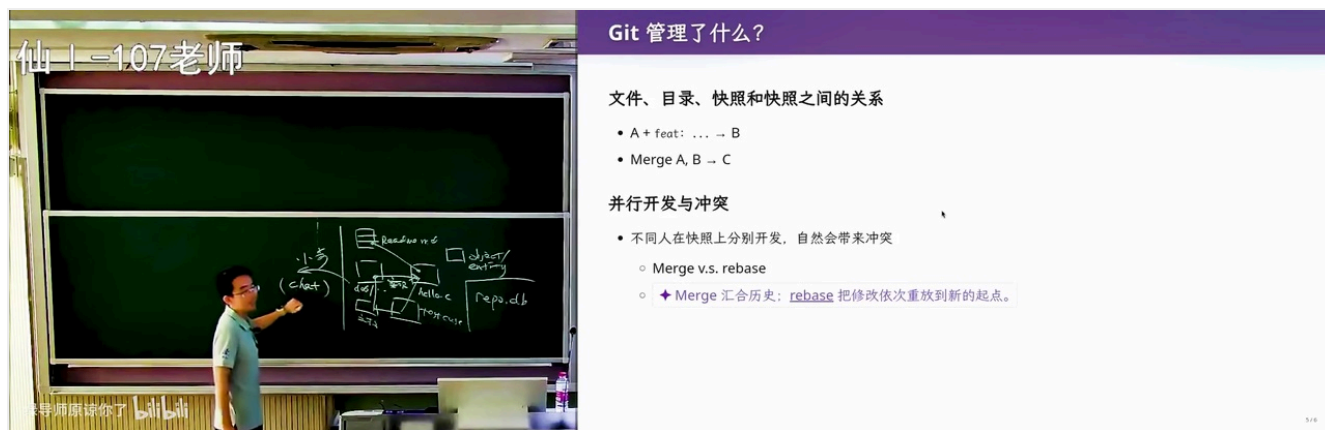
- 讲者要的是：让 AI 自动维护所有软件 artifacts 之间的一致性。
- 具体例子：
 - 代码和文档必须一致：文档说这是 linked list，实现里面就必须是 linked list。
 - 测试用例和文档必须一致：文档说要 A/B/C 三个方面的测试，test case 就必须有 A/B/C。
- 讲者强调这些 artifacts 是关联起来的。

小步修改时，大部分系统内容无关，所以让 AI 从 chat 中摘出相关部分

- 承接 Git 式的“小步前进”：当讲者谈“要做一小步”时，其实知道这一小步是干什么的。
- 因此可以跟 AI chat，例如：
 - “我现在发现它有个 bug”；
 - “我做了一个产品，今天对幻灯片样式不满意”。
- 讲者认为这就是一个小步。
- 在大型软件系统里，绝大部分内容与这一小步无关。
- 例子：看讲者的 repo（原 ASR：ripple）顶层结构，会看到：
 - 一些 assets；
 - 甚至还有 .cache，有缓存系统；
 - P1/P2/P3/P4/P5；
 - 一些脚本等。

- 讲者推论：任何软件系统，当你只做一小步时，你只需要看到这个系统的一小部分；大部分东西应该是无关的。
- 既然无关，就可以让 AI 根据 chat，把和当前任务相关的东西**摘出来**：AI 能知道现在需要的是这个文件、这个 test case、这个文档，其他东西都不需要。

第 6 部分



AI 应把与当前小步相关的东西摘出来，动态生成一个 view

- 讲者设想的场景：他从仓库里只需要 **test case** 和 **文档**，其他东西都不需要。
- 讲者的推理链：
 1. AI 有动态生成任何内容的能力；
 2. **everything is code, content as code**；
 3. code 可以表达任何东西；
 4. 所以 AI 可以根据我的 chat，生成一个和我要改的这一小步相关的 **view**（视图；原文 ASR 作 VIVO）。
- 这个想法借用了数据库里的 **view** 概念：数据就在那里，但可以创建一个不一样的查询，把它以另一种形式展示出来。讲者说，不是所有同学都学过数据库，但系统课里的这个概念在这里有用。
- 他想要看到的结果不是代码、文档、**diff**（原文 ASR 作 DF）这些东西，而是一个**可视化看板**：上面列出和这一小步相关的东西，并问“要改什么”。

小步修改是在复杂关系中从一个一致状态走到下一个版本

- 改完后，仍然经历 **V0 到 V1** 的过程，仍是一个 **repo** 的 **PR**（原文 ASR 作 rapport 点 BB）。

- 如果这一小步是“增加一种类型的测试用例”，AI 就会在相关地方加测试用例：这里加、那里加、有些地方可能要改掉。
- 也就是说，它是在各种**错综复杂的关系**中做一次小步修改，从一个一致状态到下一个版本。讲者认为，这样软件工程就“彻底变了”。
- 讲者的预测/判断：未来有可能在 AI 内部这样生产软件，从此大家再也不用看代码。

讲者讨厌现代 IDE 的默认 chat/agent 化，但仍承认这可能是路径

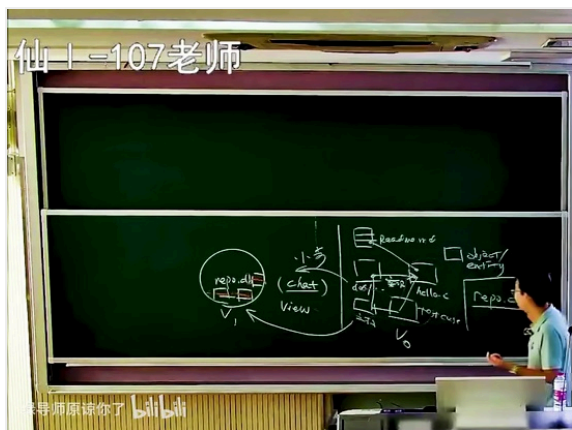
- 讲者对现在这种现代 IDE（如 **Cursor**）的态度：讨厌它默认打开就是 **agent mode**；原来熟悉的 IDE 没有了，强迫你用类似 chat 的方式沟通。
- 讲者自述相对精通“**古法编程**”，有时对代码有掌控欲；他认为这种掌控欲可能不好、可能不对。
- 但他仍认为，未来可能就是这个途径之一。

学生不必因新范式放弃经典 CS：创建这些东西用的正是经典概念

- 讲者点出学生的焦虑：如果未来一两年真有公司把这东西做出来，那大家学那么多东西还怎么办？
- 他的回应是：在思考、创造这些东西的过程中，用的都是你们学的**经典、集大成的 computer science 课程**。
- 他举自己借用的概念：
 - 数据库的 **view**；
 - 数据库的 **实体和关系**；
 - **git**：这个模型没有变；git 的长期开发经验说，要一步一步向前。
- （整理者串联）因此他想表达的是：新范式改变的是“怎么看代码、怎么改代码”的交互，不是把 CS 基础作废；view、ER、git 这些经典抽象正是 AI 内部小步生产软件的基础。

小结：讲者主张用经典 CS 概念支撑 AI 小步生成软件，让开发者面对相关视图而非全量代码。

第 7 部分



初期沿用 Git 经验：小步向前，而不是大步跳到下一状态

- 讲者提到实体和关系后，借用 Git 的模型，并说这个模型没有变。
- 讲者认为 Git 的长期开发经验是：一步一步向前。若在这个模型底下大步向前，“那就惨了”。
- 但讲者也承认另一种可能：如果模型非常非常强，面对一个非常大型的项目，也可能一步就变成下一个状态；这种做法现在还不知道怎么做。
- 因此至少在初期，讲者主张沿用过去大家积累的软件工程智慧：小步向前，然后通往“极乐世界”。讲者仍相信这个极乐世界有可能真的能到。
- 逻辑链：Git 式模型 → 一般要小步走；大步走 → 通常惨；模型极强 + 项目极大 → 可能一步跳，但方法未知；初期 → 沿用旧智慧小步走。

学生不知 rebase 是可理解的，但外部世界会把它摆到你面前

- 上上节课讲者问同学 `git rebase`，大家都不知道。一位已毕业同学 raise 了一个 valid point：学生时代大多数人不会和他人共同合作，只是做作业，所以 `rebase/merge` 用的场景很少。讲者认为这个 point 是对的：做作业时确实如此。
- 讲者的回应：如果你去外面的世界走一走，互联网开发者社区铺天盖地都是文档。你总有一天会留意别人的项目，看到 `contributing.md`。
- `contributing.md` 是仓库根目录下的一份贡献者指南，通常回答“我怎么参与这个项目”，里面会有项目总体结构、分支、提交信息、PR 的规范和流程。

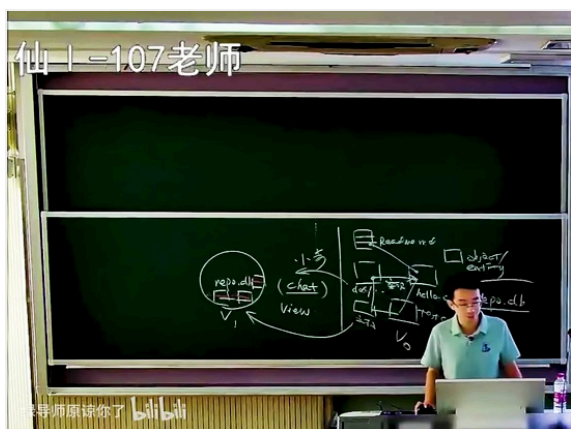
- 如果你看过很多项目的 contributing.md，随手打开一看：若里面东西都知道了，那就说明知道；若不知道，比如某一天看到一个项目要求：必须在你自己的分支上 **rebase**，禁止直接创建那种分叉的 **merge**，要保持线性历史——你就会知道 rebase 是干什么的。
- 逻辑链：学生作业少合作 → rebase/merge 场景少 → 不知 rebase 可理解；进入互联网社区 → 读 contributing.md/PR 规范 → 遇到线性历史要求 → 学会 rebase。

高中“不考不学”的思维，到大学会变成新旧人类之间的 gap

- 讲者说，现在社会变得太快，所以有一种学生——这不是批评——你们上高中时就是这样的：老师不教的，你不需要会；考试不考，你就不用学。为了升学利益最大化，这是正确的，没有错。
- 但到了大学，如果还带着这个思维，就会出现一个 **gap**：一种新人类和旧人类之间的 gap。
- 讲者相信每个同学还是可以成为“新人类”。只需要去有一些**注意力**，去注意到更多的东西。
- 讲者有一个危机感：**AI 时代的斩杀线以下的价值就全部归零**。不过讲者觉得现在的斩杀线可能还斩不到。

小结：讲者主张初期用软件工程智慧小步向前；学生不知 rebase 有现实原因，但互联网贡献者文档会补上这一课；真正要警惕的是“不考不学”思维延续成新旧人类 gap，而 AI 时代会抬高斩杀线。

第 8 部分



同学产生的疑问

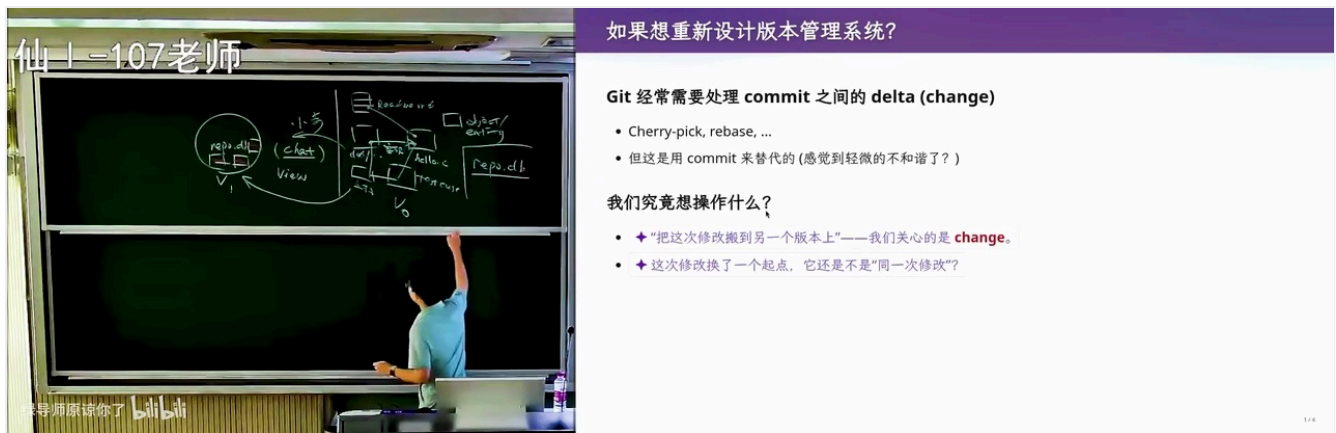
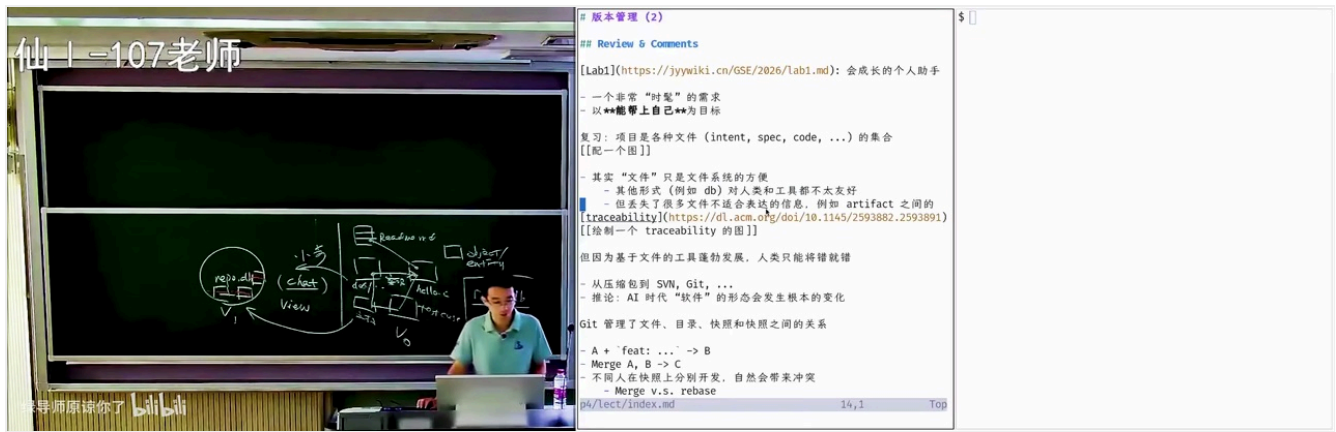
从信任老师到自己的理解

关于git rebase 0/10我想帮老师的学生答疑下，学生时代大多数人不会和他人共同合作的，大多数学生应该都是单打独斗了，用rebase merge这些场景就少很多

2020-09-08 19:25 28 0 0

你留意过别人的项目，看到过CONTRIBUTING.md，这些东西一定会出现在你的视野里，并且很容易被看懂。如果你不懂，就说明你还没开始去看别人的项目，没有去看各种开源项目的文档结构，从项目已经构建这个角度去理解“提交”建立相关性了。很长时间大家都能看懂近所有人都在写，我只要比别人不写一点就能看懂这个提交了，但AI时代是斩杀线以下全部价值为零，当然另一说，我觉得我迟早也是被斩杀，但在那之前还是得学。

- 典型的“学生思维”(信任老师)
 - 老师不教的就是我不需要会的
 - 考试不考的就是我不学用的
- 学习的目的：形成**自己的理解**(first principles)



AI 时代的斩杀线：低端智力劳动会被 agent 替代，经典课程因此更必要

讲者先提出一个危机判断：AI 时代存在一条斩杀线，斩杀线以下的价值全部归零。

- 讲者认为，现在这条斩杀线可能还斩不到 985 计算机（computer science）专业的同学。
- 但低级智力劳动危险，例如会计从业者。讲者看过报道，称中国可能有约 1000 万（具体数字讲者也不确定）会计从业者。
- 会计行业顶端的人暂时没有威胁；但底部大量从业者面临被 agent 替代斩杀的危险。
- 所以讲者主张：学生需要在操作系统、编译器、软件工程等经典课程中学经典东西，最终形成自己的理解。讲者认为软件工程里也有“软件工程的智慧”；这是他给出的评论和复习。

逻辑链：AI 有斩杀线 → 低端智力劳动在线下 → 底层会计等职业可能被 agent 替代 → 所以需要学操作系统、编译器、软件工程等经典课程，形成自己的理解。

Git 管理快照，开发要小步 commit

讲者从软件工程模型/未来畅想回到 Git 本身。

- **Git 管理的是快照**：一个目录、整个项目的快照。
- 可以在快照上开发；开发要求**小步**。
- 有 A、B 两个开发者，可以独立在这些快照上开发。
- 这个产物是 **commit**。

两路 merge 的问题：B 写的内容最后 ownership 可能归到 A

如果要把 A、B 的开发合起来：

- 讲者说，在 Git 的原始模型里，**rebase** 相对不那么提倡；Git 会让你创建一个**两路 merge**。
- 这个 merge 有 **2 个 parent**，继承两条线上的修改。
- 问题在于：如果 merge 由 A 来做，merge 里由 B 写的内容，作者会变成 A；讲者称这是一种**隐藏的作者修改**。
- 如果发生冲突，**resolve** 后提交也是由 A 完成；B 改的代码最后 **ownership** 又归到 A。
- 讲者认为这会带来一点小麻烦。

逻辑链：A、B 各自 commit → 用两路 merge 合并 → merge 由 A 执行 → B 的内容/冲突解决被算到 A 头上 → ownership 转移。

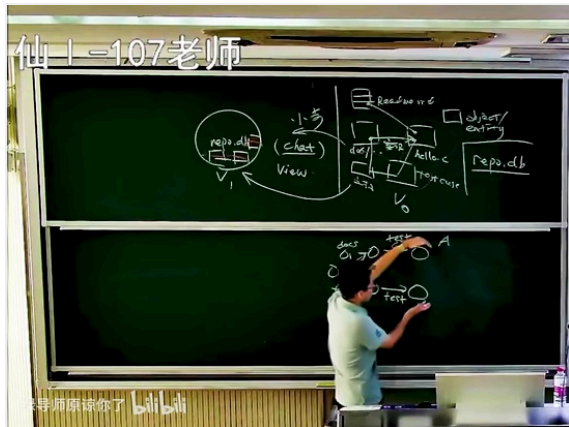
rebase 解决归属问题：不直接并线，而是把 B 的 feature 接到 A 后面

讲者画了两个历史：

- A 做了两次提交，得到两个快照，记作 (\Delta_1, \Delta_2)。
- 然后 B 开始改。
- 具体例子：A 先改文档，再加测试；B 加一个 **feature**，再加测试。
- **rebase** 不是直接创建一个把两条线路并起来的 merge commit，而是先试：B 的这个 feature 能不能在 A 的后面加上去。
- 讲者说，rebase 正是这样解决前面提到的 ownership/作者归属问题。

逻辑链：两条历史 → 不用 merge commit 直接并线 → 把 B 的 feature 试着接到 A 之后 → 避免 B 的改动在 merge 中被归到 A。

第 9 部分



如果想重新设计版本管理系统？

Git 经常需要处理 commit 之间的 delta (change)

- Cherry-pick, rebase, ...
- 但这是用 commit 来替代的 (感觉到轻微的不和谐？)

我们究竟想操作什么？

- ✦ “把这次修改搬到另一个版本上”——我们关心的是 **change**。
- ✦ 这次修改换了一个起点。它是不是“同一次修改”？

用“逐个搬 feature/test”替代 merge：目标是线性历史

讲者接着设想：不创建一个把两条线路并起来的合并提交，而是先试在 A 后面加上 **feature**。如果加上了，就算成功；成功后再把 **test** 搬过来。如果再一次成功，说明两条线路上的修改完全没有冲突，就可以一路合并过去，得到一个**线性历史**。

逻辑链：

1. 在 A 后搬入 feature。
2. 再把 test 搬到 feature 之后。
3. 若两步都无冲突 → 两条线变成一条线性历史。

风险在“不兼容”：一步冲突就进入 rebase 模式

讲者认为这个做法有很大风险：

- 搬过来的 **test** 可能与 **feature** 不兼容；
- 或者两者改了同一个地方；
- 甚至可能做了两个完全不兼容的东西。

一旦失败/冲突：

- 你需要 **resolve**；
- 会进入 **rebase** 模式；
- 讲者描述：如果进入过这个模式，会发现“瞬间自己就不知道应该怎么办”，然后打开 **agent** 问“我现在发生了什么”，**agent** 会细心解释。
- 更麻烦的是：feature 搬过来了，搬 test 时可能又失败，还得再把 test 搬过来；而且每一次都会创建一个新的提交。

merge 保留旧 commit ID；rebase 每次生成新 commit ID

每个 commit 都有 ID。

- 如果是两路 **merge**：所有原来的 commit ID 都保留，另外创建一个新的 merge commit ID。
- 如果是 **rebase**：每次 rebase 都会创建一个新的 commit，因为快照原本的 base 在这里，rebase 就是要把 base 改到新位置；base 一改，commit ID 就变。于是历史也变了，讲者说会感觉到“有一点点轻微的不一致”，好像不是完全想要这样去管理软件。

Git 的“修改”由 commit + parent 表示，没有专门的 change 对象

讲者强调：软件管理真正关心的是**修改**，希望把修改搬过来。Git 里面修改是用 **commit** 来替代的：

- 一个 commit 和它的 parent 之间的关系，就称为一个**修改**；
- Git 没有专门的 **change** 对象；
- 讲者认为，绝大部分情况都还能满足，只是“用 commit 代替 change”。

把 change 做成一等公民：下一代 VCS 的切入点

讲者接着问：如果把 **change** 也变成 **first-class citizen**，会发生什么？

他的回答：那就发明了下一代更好用的**版本控制系统**。

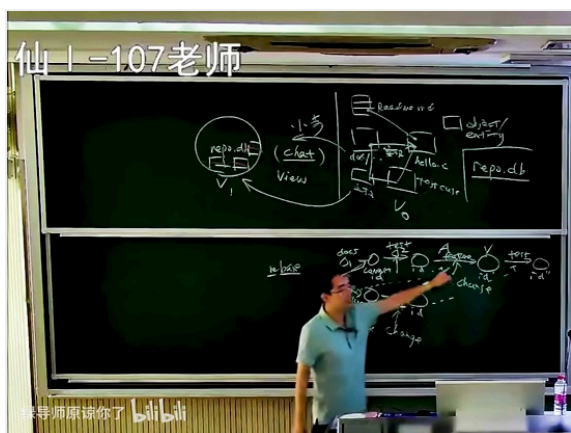
他提到一个系统（转录作“机器组”，疑为 **Jujutsu/jj**；整理者推测）：

- 与 Git 底层完全兼容；
- 可以直接当 Git 用；
- 但增加了一个 **change** 的概念；
- 每个 change 都可以 assign 一个 change (转录作“sin 一个 change”，疑为 change ID)；
- 例如 rebase 时，它会把很多 **change** 直接搬过来，而不是搬 commit。

最后他指出局限：即使把 change 做成一等公民，**change 还是可能冲突**；冲突不是靠这个模型就自动消失的（转录作“冲突是解决不了的”，此处中断）。

小结：讲者的链条是：用 rebase 逐个搬 feature/test 可换来线性历史，但代价是 commit ID 重写与冲突解决；若把 change 做成一等对象，就是下一代 VCS 的切入点，但冲突本身不会因此自动消失。

第 10 部分



把 Change 作为 First-Class Citizen

祝贺，你发明了 Jujutsu—a version control system

- Rebase 不应该让同一次 change 失去身份。
- + Commit ID 会变，change ID 延续：同一次修改，可以有不同的修订版本。[设计说明](#)

Git 视角

重写前
(旧历史仍然存在)



重写后
(新的历史)



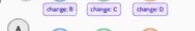
重写后得到的是一个全新的 commit；
系统本身于记录 C' 是 C 的下一版。

jj 视角

重写前
(旧历史仍然存在)



重写后
(新的历史)



重写后虽然 commit 变了，但 change 身份延续，
因此能明确知道它们分别是谁的前续修订版。

change 是对象：rebase 可搬运 change，冲突可随 change 一起带下去

- 讲者说，这种系统可以把很多 **change** 在 rebase 时直接搬过来。
- 但 change 仍可能冲突；如果在 rebase 时就要解决冲突，会很麻烦/难以继续。
- 关键原因是它有 **change object / change** 这个概念：可以把这些 change 一路带下来，生成一个已经 rebase 到目标位置的结果，即使里面引入了很多冲突。

- 它仍然沿用 Git 的冲突记录，但内部有一套自己的记录方式，把所有 change 的冲突也塞进去。于是你可以先把它们搬过来，之后再 resolve。
- 讲者结论：底层模型没有变，但用起来舒服很多。
- **整理者串联**：change 成为一等对象 → rebase 不必被冲突阻塞，而是把“带冲突的 change”先搬过去 → 冲突处理从“当场解决”变成“稍后 resolve”。

Git 最讨厌的场景：改到一半要切任务，只能 index + stash，stash 还会堆栈

讲者用上课会讲的例子说明 **staging area** 的痛点：

1. 工作区有一个 repo，你在改 **hello.c**，改到一半。
 2. 此时项目还不能编译、不能运行，测试也通不过。
 3. 导师突然打电话：有一个很高优先级的项目 bug，客户马上要交差，要求一分钟修好。
 4. 讲者说，这时你不应该 commit。
 5. 于是需要把当前改动加到 **index**，然后 **git stash**，把状态压栈。
 6. 你去修原来的 bug，刚修一半，导师又来电话：这个别修了，还有优先级更高的 bug，去修那个。
 7. 你又要再 **git stash**；而 **git stash** 是一个栈，里面会积很多提交，你需要管理这个 stack。
- 讲者认为这是一个比较讨厌的操作。
 - **整理者串联**：痛点是频繁任务切换时，临时工作状态被 staging area 和 stash 栈管理复杂化。

把工作区编辑本身看成 change：自动提交，但用当前修改替换上一次提交

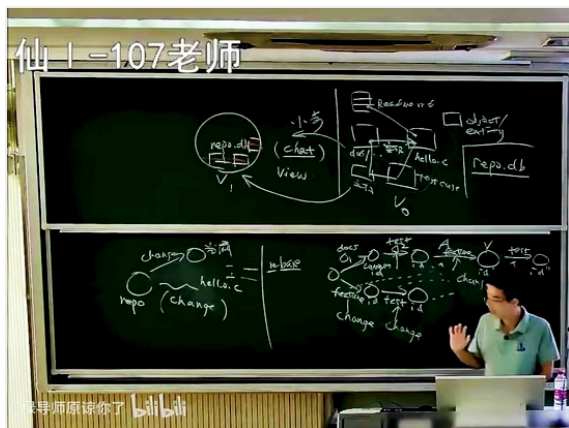
- 讲者反问：如果你想象在任何时候、随时随地都在编辑这个 repo，这不就是一个 change 吗？
- 你现在把 **hello.c** 改一改，加一行、减一行，都是 change。
- 所以 **Jujutsu**（原转写为“JJS/jj”）看起来在做的是这样的事：
 - 你但凡改了一个字节，它就立即做一个 **git commit**；

- 但这个 commit 不会产生很多线性的历史；
- 它先把上一次的 commit 抹掉；
- 然后用当前修改去替代那个提交；
- 也就是说，你永远处于一个 **change 提交的状态**，当前 snapshot 永远是一个提交状态。
- 因此，你连 **staging area** 也不需要了。讲者认为这非常舒服。

从“change 作为 first-class citizen”出发，可以往后推出自己的 VCS

- 讲者说：从“我要把 **change 作为 first-class citizen**”开始往后推，你就可以发明自己的版本管理系统。
- 但这不是它的全部；讲者提到它还有一些别的特性，此处未展开。
- **整理者串联**：先确立 change 是一等对象，再推出 rebase 搬运与冲突延后、无 staging area 的工作流。

第 11 部分



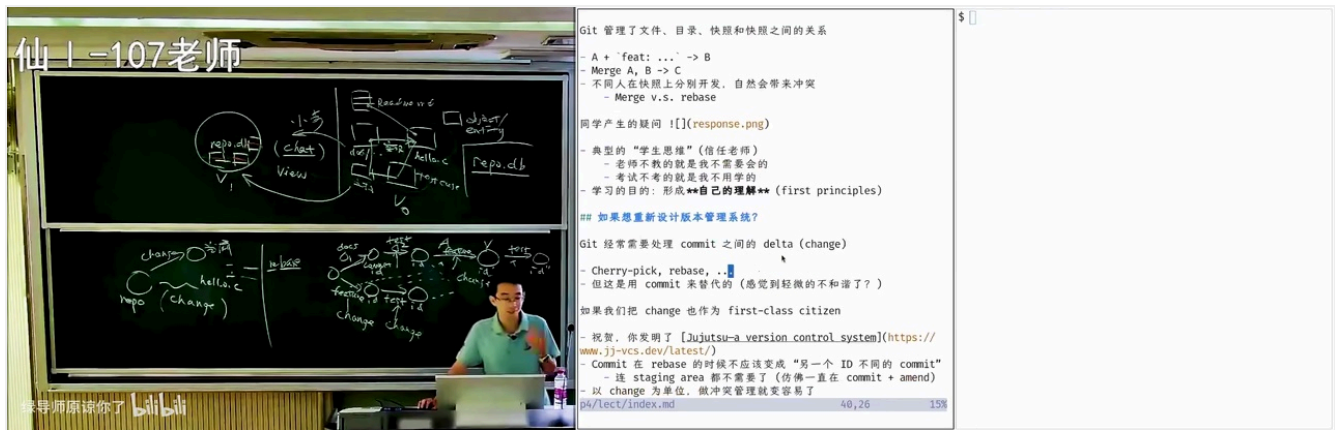
让未完成的工作也有位置

连 staging area 都不需要了

- 仿佛一直在 commit + amend。
 - ✦ 工作副本本身就是一个 commit；多数 jj 命令会自动记录文件修改，更新这个 commit。[工作副本](#)

冲突直接存进 commit (killer feature!)

- Rebase 可以直接完成，之后再统一处理冲突。
- ✦ 保存的是冲突的逻辑表示：带着冲突的 commit 仍能继续 rebase、merge。[冲突管理](#)
- ✦ 设计启发：把“尚未解决”表达成一种可保存、可继续操作的状态。



从 change 是一等公民出发，能发明自己的版本管理系统

- 讲者提出：把 **change as first-class citizen** 作为起点，再往后推，就能发明自己的版本管理系统。
- 但这不是全部；还有其他路线也能推导出别的设计。
- 讲者建议适当质疑 Git：Git 有没有哪里做得不够好？如果可以做得更好，还能怎样设计？讲者认为这些设计非常有趣。
- 学习路径：讲者已经给了 motivation；学生可以把最重要的 prompt 交给 agent，例如“我要学 Jujutsu（ASR 作“句句词/GG词”，按上下文应指 Jujutsu），我想从 change 开始理解”。

Jujutsu 的 tutorial 一上来就说：没有 staging area，版本管理更自然

- 讲者说，看它的 tutorial，一开始就会告诉你它没有 **staging area**。
- 因此管理版本更自然：首先对人类更友好；讲者也认为它可能对 agent 稍微更友好一点。
- （整理者串联）如果 change 是一等公民，就不必把“先 stage、再 commit”当成必经流程；这正对应讲者举的“没有 staging area”作为更自然管理的例子。

肌肉记忆造成迁移成本：老一代被 Git/SVN 锁住，AI-native 一代可直接用更对的设计

- 讲者认为，AI-native 一代能“淘汰”上一代，原因之一是：讲者这一代已形成 **Git 肌肉记忆**，更老一代已形成 **SVN 肌肉记忆**。
- 一旦形成肌肉记忆，要把习惯掰过来很困难。
- 讲者即使能理解 Jujutsu 的概念，甚至可能比初学者理解得更好，能从 **first principles** 思考，相当于 **virtually reimplement** 这个系统；但也没有足够时间再训练另一套肌肉记忆。

- 学生的优势正好相反：现在还没形成旧记忆时，就直接用“对的东西”。讲者说人类历史是积累的：SVN 实在用不下去了才用 Git；Git 的缺点被发现以后，有了 Jujutsu（ASR 作“GG子”）。
- 讲者建议：先用肌肉记忆记住一个最正确的设计，再回头理解这个设计。

回头再看旧设计：Git 先进，但也不是没有问题

- 讲者描述认知变化：Git 出现时大家觉得它很先进，因为它是 **persistent data structure**；而 SVN 从一开始就在某条设计路线上走错了（原文作“纯 DF”，未展开）。
- 到现在再回看，会发现 Git 也犯了一些设计问题。
- 讲者认为原因之一：Git 最早没有想过 **rebase** 会有这么大的 impact；当时有 **merge** 就已经很好，能解决他手上的问题，所以可能没有把概念模型想得更清楚。
- 讲者把这些称为有意思的 comments，也是学习建议：学东西时，尤其在 AI 辅助下，可以不断往前走。
- 讲者最后提醒：这门课不是软件工程。

第 12 部分



Git 管理了文件、目录、快照和快照之间的关系

- A + 'feat: ...' -> B
- Merge A, B -> C
- 不同人在快照上分别开发，自然会带来冲突
- Merge v.s. rebase

同学产生的疑问

- 典型的“学生思维”（信任老师）
 - 老师不教的就是我不需要会的
 - 考试不考的就是我不需要学的
- 学习的目的：形成**自己的理解** (first principles)

如果重新设计版本管理系统?

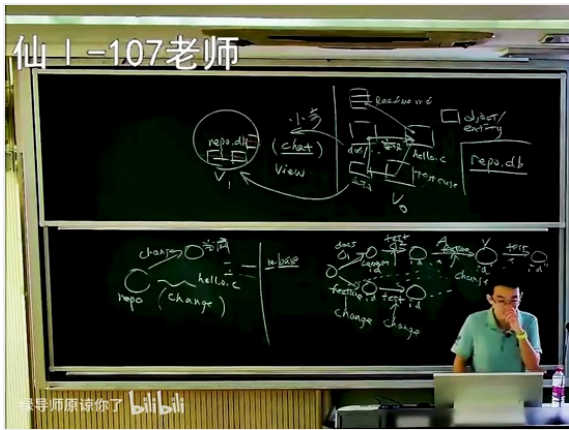
Git 经常需要处理 commit 之间的 delta (change)

- Cherry-pick, rebase, ...
- 但是用 commit 来替代的（感觉到轻微的不和谐了？）

如果我们把 change 也作为 first-class citizen

- 祝贺，你发明了 [Jujutsu-a version control system](https://www.jj-vcs.dev/latest/)
- Commit 在 rebase 的时候不应该变成“另一个 ID 不同的 commit”
 - 连 staging area 都不需要了（仿佛一直在 commit + amend）
- 以 change 为单位，做冲突管理就容易了

66/lect/index.md 36,43-30 15%



Agent 时代，再次重新设计

人类是“Single-Threaded”生物

- 我们只有一份 I/O 设备：一双手、一张嘴、一对眼睛

Git 基本够一个人用了

- stash: 把手头的工作放一边，先处理另一件事
- branch: 保留不同方向的尝试，切换着推进
- Jujutsu 就更好用了

课程定位：生成式软件工程要在 agent 时代重新设计版本管理系统

- 讲者强调：这门课不是软件工程，而是生成式软件工程。
- 真正要做的事：在 agent 时代再次重新设计版本管理系统。
- 当然可以让 agent 去用 Git，也可以让 agent 去用 Jujutsu（原文音近“JUJS/JUSS”）。
- 讲者认为：Jujutsu 对人类来说是大进步；但对 agent 来说进步可能相对小，因为 agent 似乎很会 rebase 这类操作。
- 问题链条：SVN 有局限 → Git 有局限 → agent 时代的局限是什么、要破掉什么？

旧 VCS 的隐含前提：使用者是人类

- 讲者指出：此前所有 version control system (VCS) 都有一个假设——使用者是人类。
- 人类的关键特点：人类是**单线程 (single-threaded)** 的生物。
- 因此，若要为 agent 重新设计 VCS，就必须检查这个“用户=人类”的前提。

操作系统并发类比：read 与 write 不能同时发生

- 讲者用并发程序打比方：把线程想象成一个人，线程可以访问共享内存。
- 共享内存的基本操作是 load/store，也即 read/write。
- 读一个变量，是把 X 读到临时变量；写一个变量，是写 X 等于某个值。
- 这两个操作不能同时进行；若要求同时，就需要原子指令。
- 讲者把人的行为类比成共享内存访问：
 - 人观察世界像 read：手要抱在后面，只能看。
 - 等看完后，比如看到某位同学在这里，想对他施加效果，比如用粉笔导弹射击他。
 - 但真正要 write、要动手时，必须“把眼睛闭起来”。
 - 闭眼扔的时候，之前读到的值已经失效了，是上一个时刻读到的旧值。

- 中间可能发生 CPU 切换、中断、另一个线程上来执行；恢复后寄存器里还是上次读到的 X。
- 于是粉笔扔出去很可能就是错的。
- 讲者用这个例子说明：人和线程一样，受“读—写不能同时、读到的值会过期”的限制。

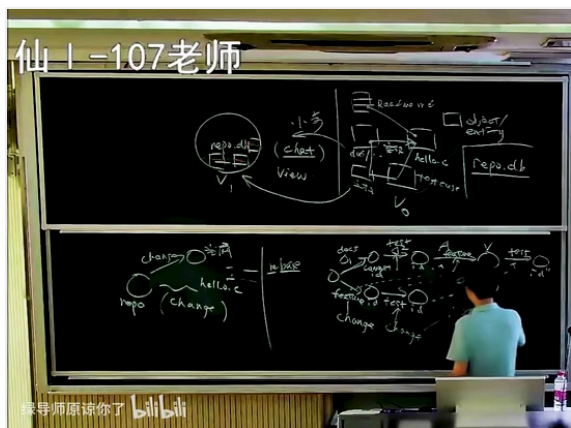
单线程人类导致代码变更以小时计

- 讲者进一步说：人之间有无可逾越的**物理屏障**。
- 你不能直接理解我的思想，我也不能直接理解你的思想；每个人是独立个体，由分子连在一起。
- 人只有一套 I/O 设备：一双手、一张嘴、一对眼睛。
- 所以以这种方式编辑代码库、产生 change，粒度是**以小时为单位**。
- 一个软件公司负责一个模块的开发，可能也就十几个人；十几个人已经是一个不小的团队。
- 这十几个人每个人仍然以小时为单位开发。

逻辑链

1. 旧 VCS 默认用户是人类。
2. 人类是单线程的：read 与 write 不能同时完成，观察和行动之间状态会过期。
3. 因此人类产生代码 change 的速度和并发度受生理、物理限制，粒度以小时计，团队协作也限于十几人级别。
4. （整理者串联）若 agent 不是这种单线程人类，继续沿用旧 VCS 就会保留“人类用户”假设带来的局限；这正是讲者要在 agent 时代重新设计版本管理系统的理由。

第 13 部分



Agent 时代，再次重新设计版本管理系统

人类是“Single-Threaded”生物

- 我们只有一份 I/O 设备：一双手、一张嘴、一对眼睛

Git 基本够一个人用了

- stash: 把手头的工作放一边，先处理另一件事
- branch: 保留不同方向的尝试，切换着推进
- Jujutsu 就更好用了

人类提交速度受沟通速度限制，所以 Git/stash 为人设计是合理的

讲者的链条：十几人团队、小时级开发 → 最快约 10 分钟一次提交 → 这比人与人沟通慢很多 → 因此 stash 不必太深，Git 为人设计是合理的。

- 一个软件公司负责一个模块开发，可能也就十几个人，已经是一个不小的团队。
- 这十几个人每个人以小时为单位开发；最多也就是以 10 分钟为单位产生一个提交。
- 讲者认为：以 10 分钟为单位产生提交，意味着这个速度比人与人之间沟通的速度要慢很多。
- 例子：
 - 尤其一个 team，工位都在一起；研究生的工位也都在一起。
 - 你要写一个东西要一两个小时，写之前肯定会先跟师兄说一声：“师兄，我要干这个行不行？”
 - 十几秒、二十秒、30 秒，师兄说行，“你去改吧，我们现在不改”。
 - 如果去厂里打工，厂里可能每天早上开会，leader 把任务分出来：今天要干什么，你做什么。
 - 你在半小时里划分一个 **scope**，把项目切开，领走其中一部分，然后去产生提交。
- 讲者称这是“古法编程”。

古法编程下，stash 是合理设计

- 在古法编程时代，**stash** 是一个合理设计。
- 因为导师给你打电话这件事可能发生；但你在修到一半的时候，导师再给你打电话，这件事不太可能发生，概率很低。
- 绝大部分时候，你的 stash，也就是 **stash stack**，里面只有一层。
- 如果你有好多层，收起来其实挺麻烦。
- 还有一点麻烦：你还需要去 **rebase stash**。
- 所以你会尽可能保证 stash 不要太深。
- 因为你可以拒绝一个工作，或者说：“我手上有一个更高优先级的工作，现在对不起，我不能干这个事。”
- 等你干完了以后，你自己可以排一个队列。
- 讲者结论：**Git 是给人用的**。

Agent 时代：单个 agent 以秒级产生原子提交，人设计的 Git/[GG] 跟不上

讲者的链条：单个 agent 生成速度快于人 → 以分钟级甚至秒级工作 → 可以产生原子提交 → 可以轻易得到 10 倍、100 倍 agent team → Git/[GG] 这种为人设计的模型跟不上。

- 但是在 agent 的时代，这件事变了。
- 如果是一个 **agent team**，单个 agent 首先生成 **AI slop**（原音近似 AISLOFT）的速度就远远快于人。
- 它生成 AI slop 以分钟级为单位，甚至更快。
- 有时候你让它 fix 一个东西，在上下文热的时候可能就只要几十秒；像 [DPCV4flash] 这样的模型，干活时几秒钟就做了。
- 当 agent 以秒级工作时，它可以产生一个**原子提交**。
- 并且你可以轻易得到一个 10 倍、100 倍的 agent team。
- 因此，Git 或 [GG] 这样与 Git 兼容、为人设计的模型，就有点跟不上这个速度。

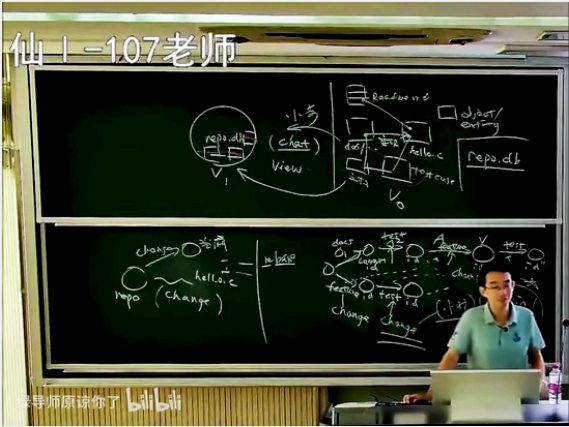
测试门控只能放缓，不能逆转 agent 加速开发的事实

- 就算是大型项目，它也有一些门控：当你写完、改完这个部分以后，必须要经历一个测试流水线。
- 测试本身可能就要运行几分钟，这可以放缓 agent 提交的速度。
- 但讲者认为毫无疑问：**agent 加速了软件开发的过程**。

新问题：如何为 agent 设计版本管理系统？

- 如果 agent 加速了软件开发的过程，我们应该怎样为 agent 设计版本管理系统？
 - 讲者提出，首先可以让 agent 沿用人的模型。
 - 具体想象：每个 agent 都有一个自己的开发计算机。
-

第 14 部分



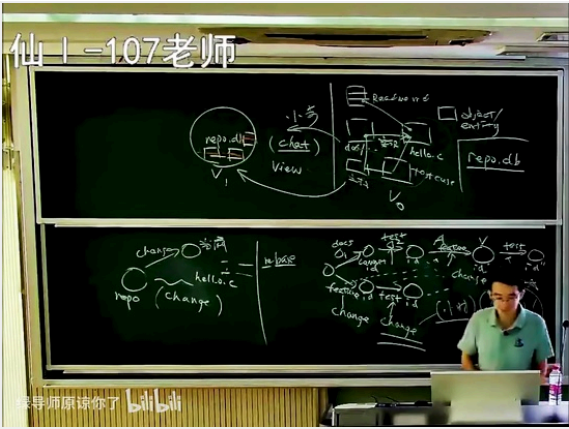
把每个 Agent 当成独立开发者

每个 Agent 有自己的开发计算机，用 Remote 同步

- 这个流程能不能放到本地？
- Yes! 我们有了 `git worktree`

一个仓库，多个工作目录

- 每个 worktree 有自己的工作文件、HEAD 和 index (暂存区)
- 共享对象库和分支引用：提交在本地直接可见，无需互相 push/pull
- 给 Agent A 创建分支和工作目录：`git worktree add -b agent-a ../gense-agent-a`
- 给 Agent B 创建分支和工作目录：`git worktree add -b agent-b ../gense-agent-b`
- 各自在自己的目录干活：修改仍需 merge/rebase 才会汇合



Thinking...

GenSE 仓库里没有 CONTRIBUTING.md (我在项目根、docs/ 和 wiki 里都搜了，一个都没有)。

它的通行含义是：仓库根目录下的一份“贡献者指南”，通常回答“我怎么参与这个项目”的问题，典型内容：

- 环境搭建、依赖安装、构建/测试命令
- 分支、提交信息、PR 的规范与流程
- 代码风格、目录结构约定、评审要求
- 行为准则（有时折到 CODE_OF_CONDUCT.md）

对比一下本项目现在的实际情况——这类信息并没有集中在一处，而是散在：

文件	承担的角色
AGENTS.md / CLAUDE.md	给 AI Agent (也给人) 的项目约定、使用者设定
README.md	构建方式：make slides N, 依赖、导出行为
docs/slice-format.md	切片原稿的格式规范
.agents/skills/*SKILL.md	各类具体工作流 (compile, make-slides, transcribe...)
git log	实际在用的提交信息风格 (中文 + 范围前缀，如 Add p5 lect 架构 skeleton)

所以如果这门课要帮助教/协作者一个入口，写一份 CONTRIBUTING.md 会是个合理补充：把 README 的构建命令、docs/slice-format.md 的格式约束，以及提交信息风格（照抄现有 log 的中文风格）汇总成“新闻学第一天要读的东西”，需要的话我可以直接起草。

逻辑链（讲者的推进）

1. agent 先沿用人的模型：把人类 **single developer（单人开发者）** 和 agent 做严格——对应。
2. 每个 agent 有自己的开发计算机/容器 → 放到远程 Git 仓库 → 从 **tmux** 接收命令 → 收到就开干。
3. 两个 tmux session 可位于不同机器/容器，物理隔离、互相看不到 → 模拟多个独立开发者。
4. 讲者提出：AI agent 变快时，要想如果指挥发生一点点差错会怎样；随后转到流行的 **监工 agent + 多个子 agent** 范式。
5. 为了最大化模型智力、防止 AI 生成不想要的东西，用 **Codex** 设计、**Claude** review，让它们互相打架并达成一致；主 agent 协调、看报告、inject prompt → 多 agent 转起来。
6. 同一 Git 仓库会有麻烦；不同 **repo** 则可合并、拉取、修冲突、通知 master → 可立即构建多 agent 系统。

7. **git worktree** 是 workaround；讲者以提问方式引出它，并强调提问是关键。

单人开发者类比：— agent、一台开发机、一个远程仓库

- 讲者主张：让 agent 沿用人的模型，把 **single developer** 的人类和 agent 做严格一一对应的类比。
- 每个 agent 都要有一台自己的**开发计算机**，可以是一个**容器**。
- 把它放到一个远程 Git 仓库上（转录：git 点 NHU 点 E 点 CN；具体域名不确定）。
- 直接 instruct agent：我的 remote 仓库是在 Git 的；你现在就是一个单人开发者；你从 **tmux** 里接收命令；收到命令就可以开干。
- 讲者结论：这件事非常容易实现。

演示：两个隔离 tmux session 各自开干

- 讲者用类似 **fancy project** 的演示，直接让 agent 开始工作。
- 两个 **tmux session** 可以位于两台不同机器或不同容器，**物理隔离**；它们看不到彼此，也看不到“世界上其他的人”。
- 演示中故意用 **Master / Slave** 命名（Slave 转录作 Slav）：讲者当 **master**，给每个 agent 起 coding agent 名，然后让它们工作。
- （整理者串联）这种物理隔离让多个“单人开发者”可以并行而不互相干扰。

多 agent 范式：监工 agent + 多个子 agent

- 讲者提醒：当 AI agent 变快时，如果指挥发生一点点差错，甚至只是指挥本身，就会引出问题。
- 现在流行一个 **paradigm**：一个**监工/supervisor agent** + 多个子 agent。
- 为了最大化模型智力、防止 AI 生成你不想要的东西，可以放两个较强模型：
 - **Codex** 来设计；
 - **Claude** 来 review；
 - 让它们互相打架，然后达成一致。
- 实现上只要开两个 **tmux session** 即可。
- 主 agent 负责协调：观看它们的输出或总结报告，然后往里 **inject prompt**。这样多 agent 就转起来了。
- 讲者查看 **report status**。

并行冲突：同一 Git 仓库麻烦，不同 repo 可行

- 观察：如果大家工作在同一个 Git 仓库上，就会有一些麻烦。
- 如果工作在不同的 repo 上，就完全可以：
 - 可以合并；
 - 可以去远端拉取别人的提交；
 - 冲突了就自己修复；
 - 必要时甚至可以给 master agent 发消息。
- 因此可以立即构建出一个多 agent 系统。

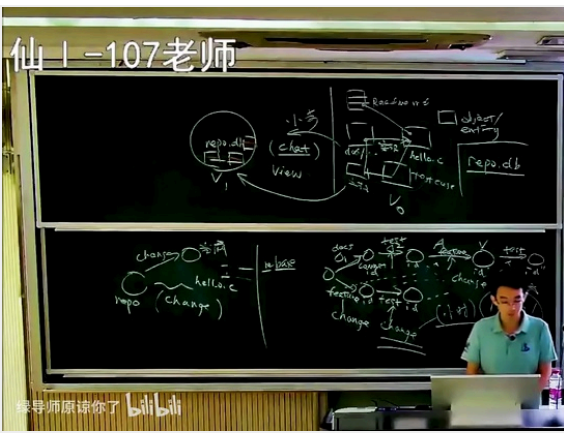
workaround：git worktree 与“提问是关键”

- 讲者提到今天也有一些 workaround，例如 Git 的 worktree 功能。
- 他给出提问示例：我的老师说 git worktree 可以改进这种并行工作方式，那么能不能让这两个 tmux session 都工作在 worktree 里？
- 讲者强调：提问是关键。你们可以在各种信息/场合提问，甚至比如今天早上走到教室的时候（转录后半不完整）。

小结：agent 按单人开发者隔离部署，多 agent 靠 tmux/supervisor 协调；同仓库冲突是障碍，git worktree 是讲者提到的 workaround。

第 15 部分

仙 I -107老师



			op.py
/tmp/fancy-slaves/slop-b	✗ 非仓库	-	slop.py 2096 行
/tmp/fancy-project (本工头)	✓ 已 init	master, 0 提交	空

要点：只有 A 建了 repo (且干净，单提交)；两坨 slop 都还没纳入版本控制，散在各目录里。若要让工头统一收编，下一步就是给 slop 建仓/提交。

Recap：用一条 for 循环对多个沙箱目录做「有 .git 就 git log/status，否则标注非仓库」的活。比这个步骤更快拿到全局视图：同时用 capture-pane | grep -c working 确认 slave 已停工，保证看到的是稳定快照而非生成中。

我的老师说，git worktree可以改进这样并行的工作方式。

/tmp/fancy-project (master)	+5.3k	44.0k	R68k CH97.8% \$0.002 0.9%/1...

slop_a.py	523 4	生成的 slop 模块 (240 个函数 + 20 个 Handler 类 + 分发表)	
gen_slop.py	~90	生成器 (固定 seed=2026, 可复现)	

- 行数超预期 1000, 且 import slop_a; slop_a.main() 可正常运行，不是一堆死代码。

- 内容刻意低信号：重复的 acc += (idx * i) % 97, scratch.append(...), 千篇一律的 docstring 与 try/except 包裹，作为 "AI 味代码" 样本。

Recap：用固定随机种子 + 模板化生成器批量产出“像模像样但低信息”的代码，既保证可运行 (生成后立即 import 验证) 又能精确控制规模；随机种子让 slop 可复现，方便后续做 diff/评审实验。

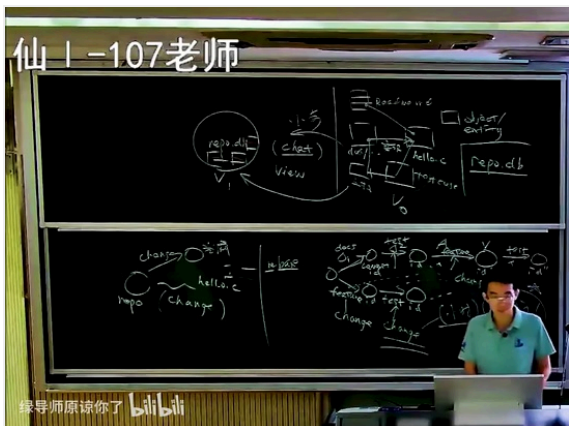
```
~/Projects/GenSE (main)
t1.2k 42.5k R23k CH97.3% $0.001 0.6%/1...
[slave-a] 81ps>n - GenSE 18:59 15-Sep-24
```

```
~/Projects/GenSE (main)
t1.0k 418k R68k CH99.2% $0.005 2.1%/1...
[slave-b] 81ps>n - GenSE 18:59 15-Sep-24
```

达 SAD)、伪 ML、伪加密、CLI、自测试 - 已验证：self.test() 29 项全通过，CLI 各子命令可运行

```
'''bash
python3
/tmp/fancy-slaves/slop-b/slop.py
--version
python3
/tmp/fancy-slaves/slop-b/slop.py hello
--name slop --times 2
python3
/tmp/fancy-slaves/slop-b/slop.py slug
"Hello, GenSE 2026!"
python3
/tmp/fancy-slaves/slop-b/slop.py bench
--size 100
...'''
```

Recap：用“反面教材”的方式产出 >2000 行可运行但冗余冗余的 slop (工厂的工厂、永远转不到 READY 的状态机、假 ML/加密、自测自嗨)，既能满足代码量要求，又能当课堂素材演示“企业级过度设计”。



把每个 Agent 当成独立开发者

每个 Agent 有自己的开发计算机，用 Remote 同步

- 这个流程能不能放到本地？
- Yes! 我们有了 `git worktree`

一个仓库，多个工作目录

- 每个 worktree 有自己的工作文件、HEAD 和 index (暂存区)
- 共享对象库和分支引用：提交在本地直接可见，无需互相 push/pull
- 给 Agent A 创建分支和工作目录：`git worktree add -b agent-a ../gense-agent-a`
- 给 Agent B 创建分支和工作目录：`git worktree add -b agent-b ../gense-agent-b`
- 各自在自己的目录干活：修改仍需 merge/rebase 才会汇合

3/6

能让多个 tmux sessions 都工作在 worktree 吗：讲者把“提问”当作关键

- 讲者开场问：能不能让两个 `tmux sessions`（转写里作 Mark sessions）都工作在 `worktree`？他认为“提问”本身是关键。
- 现实入口：比如早上走到教室时看微信公众号，看到有人提到 `git worktree` 或某种 `sandbox container`，就可以立刻让 `agent` 帮你试，自己观察它在做什么。
- Demo：agent 确实在写脚本，并产出了 `patch`（转写作 pad）；讲者认为这不是一个相对困难的任务，最后 `worktree` 应该是可用的。
- （整理者串联）多个 `agent/tmux session` 要并行提交，就需要隔离工作区；`worktree` 正好用共享仓库提供隔离工作树。

worktree 的机制：`.git` 可以是目录，也可以只是文本文件

- `git worktree` 允许创建一个 **工作树**；讲者说可想象成连接到同一个 **Git 仓库**，在里面能做独立提交，而且是在隔离环境里提交。
- 普通仓库里，比如“我的 JS1”，`.git` 是目录，里面有 `logs`、`objects` 等；讲者提到有三种类型的对象。
- `worktree` 里的 `.git` 点开是普通文本文件；文件内容类似 `gitdir:`（转写作 GDIR），指向一个 **worktree 元数据目录**（转写里作 **VTREE**）。这个位置仍在文件系统里。
- 这个目录是一个小型 `.git`，或者说微型 Git 目录，依然有 `HEAD`、`index`、`locks` 等；但它依附于原有的 `.git` 目录而生。
- 讲者强调：Git 最大程度复用了操作系统里的机制。

worktree 在 AI 时代有用：本地提交互相可见，不必互相 pull/push

- 讲者指出，worktree 这个“依附原 Git 目录衍生出的特性”，在 AI 时代突然显得有用。
- 因为多个 worktree/agent 的提交可以在本地互相可见，不需要互相 pull 和 push。
- 替代方案：开好几台虚拟机，每台虚拟机里放一个 agent，再直接往 tmux 里注入；讲者说这完全可行。
- 但 worktree 提供了更轻量的合作方式。

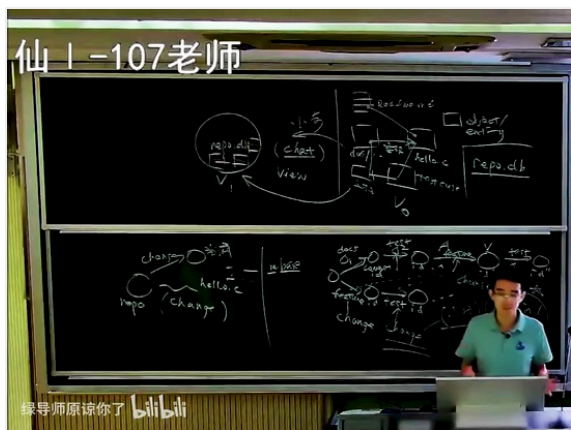
worktree 的边界：冲突仍要 merge/rebase

- 如果有冲突的修改，仍然需要 merge 和 rebase 才能合并。
- 讲者说“这样性能会更高一点”（转写如此；可能指并行/性能）。

回到 agent team / agent swarm：为什么需要版本控制

- 讲者把话题拉回：如果我们有 agent team 或 agent swarm，除了 worktree 这样的机制，还要回头想：为什么需要 版本控制 / 版本管理？
 - 他的解释是：因为人的限制。人的 commit 速度很慢——一个小的团队，哪怕是一个团队，也要约 10 分钟才产生一个 commit。
 - 逻辑链：人的 commit 速度慢（约 10 分钟/commit）→ 人之间需要同步 → 每天早上要开一次会议，同步各人的进展，再把任务分下去 → 这些人类协作限制塑造了版本控制/版本管理工具。
 - 讲者明确把“为什么需要版本控制”归因到人的限制与 communication。
 - 他进一步说：“而且就算是人做了这样的沟通……”（此处结束）。
-

第 16 部分



回到为什么需要版本控制

人的限制，塑造了工具

- 人的生产效率有限
- 人之间有 communication gap：实际上沟通效率极低
- 人就算沟通了，也可能各种出错
 - 快照、小步前进：bisect、blame.....

如果没有 Agent 能被开除、能坐牢？

我们也不要 blame，只要把问题默默修了就好？

- ✦ 历史的价值可以从“谁写错了”转向“哪次变化破坏了什么，怎样恢复”
- ✦ 跨领域先例：[NASA 航空安全报告系统](#)通过自愿、保密、非惩罚性的报告收集安全隐患，帮助防止重演

沟通成本随人数上升，团队不能按人数线性放大

- 人与人之间的 communication 会变成日常会议：每天早上开一次会，同步各人进展，再把任务分下去。
- 讲者强调，即使人做了沟通，也不保证讲得明白：室友来请教数学题，你明明会做，却可能怎么都没法给他讲明白；除非你是很好的老师，能共情、站在别人的角度思考。所以沟通成本很高。
- 由此解释大厂/小厂、尤其初创公司或初创团队为何效率更高：软件系统变大、人变多后，沟通效率下降，产出速度也下降。
- 讲者给出数量级判断：100 人团队不可能达到 10 人团队效率的 10 倍；有 2—3 倍就已经很不错。
- 所以团队没有办法无限变大。（整理者串联） 这正是要引入 Git 各种 best practice 的背景。

Git 最佳实践：快照 + 小步 commit，让问题可追溯

- 使用 快照 (snapshot)，小步前进。
- 小到改一个 title 也要做一次 commit；做任何事都做一个 commit，这些小的 commit 每个都可以追溯。
- 一旦出问题，需要回溯到某个人做那次 commit 的时间；那个人还有记忆。开发者会回想：当天早上开了会，收到这个需求；做需求时沟通出了问题；大家找到当时的沟通记录文档，再判断这个地方应该怎么修正。

- 所以有 **git bisect**（原音近似“get it by sec”）、**git blame** 这类专门为人设计的特性。

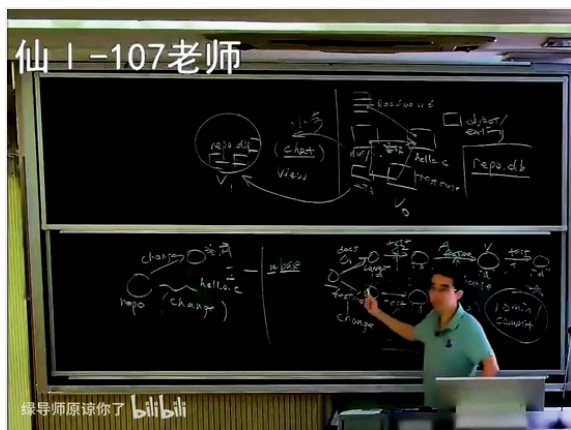
Agent 团队里 blame 失去意义：不再追责，只需修错和前进

- 讲者反问：如果由 **agent team** 或 **agent swarm** 来实现项目，还需要 **git blame** 吗？
- 因为人可以 blame：你被 blame 多了，就可能被开掉；实习生就会背锅。
- 但如果 agent 没有“坐牢/被开除”这种承担后果的能力，模型就变了：讲者认为，**再也不需要 blame**，项目只要前进就可以。
- 前进时有时也不需要知道到底是哪一个提交引入错误；只要知道这个地方错了，不必再管历史上怎么造成，直接把错误修了就行。
- 讲者提到 **agent 之间的 communication**，并强调更好玩的点：每个 **agent** 都是相同模型，是完全相同的克隆体。同一件事让它们做，大概率会收敛到相同结果；虽然模型仍有不确定性，同样 prompt 做同一件事会有细微差别，但最终方向性的东西可能收敛到相同结果。
- 因此项目被讲者描述为一个**永远向前进**的过程。**（整理者串联）** 这等于把 Git 的角色从人类追责部分转向状态保存与前进。

不需要 blame ≠ 不需要快照：方向错了仍要能回退

- 讲者明确说，**快照肯定还是需要的**。
 - 例如方向走错时：如果没有保留历史上的某个 **snapshot**，已经往前走了很多之后再回去，代价与“一点一点往前修”相比会成为问题；所以保留快照仍然必要。
 - **（整理者串联）** 这里的关键不是回溯责任，而是保留可回退的历史状态。
-

第 17 部分



回到为什么需要版本控制

人的限制，塑造了工具

- 人的生产效率有限
- 人之间有 communication gap：实际上沟通效率极低
- 人就算沟通了，也可能各种出错
 - 快照、小步前进：bisect、blame.....

如果没有 Agent 能被开除、能坐牢？

我们再也**不要 blame**，只要把问题默默修了就好？

- ✦ 历史的价值可以从“谁写错了”转向“哪次变化破坏了什么，怎样恢复”
- ✦ 跨领域先例：[NASA 航空安全报告系统](#)通过自愿、保密、非惩罚性的报告收集安全隐患，帮助防止重演

快照要留，但历史的价值从“追责”变成“继续前进”

- 讲者认为：一下子往前走了很多再回去，代价可能比一点一点往前修更大；所以保留快照仍然需要。
- 但主线策略是：不断大步向前，几乎永远不要看过去；Git 的 commit 历史只留下一些 **lessons**，或记录项目经历过怎样的演化。
- 因此动作是“往前走、合并、合并、合并”。
- 逻辑链：大步前进 → 回退/修复代价高 → 仍要保留快照 → 但 commit 历史主要不是用来回头追责 → 历史价值从“谁写错了”变成“我怎么把它修好”。
- 只要保存了快照，能知道“从现在开始怎样往前走”就够了。

Git 模型像数据库并发控制：事务里可以混任意代码

- 讲者说，这里又回到计算机科学知识：操作系统里的**并发**、数据库里的**并发控制**。
- 现在的 Git 模型更像数据库中的并发控制。
- 例子：两个 SQL 都可以是一个 **transaction**，而且可以是很大的 transaction，中间可以混任何代码。
- 一个事务先 `SELECT` 读一下；另一个事务也可以 `SELECT`；读完后可以做代码计算，例如 Python 或 JavaScript 代码。
- 这些代码是**图灵完备**的：你永远不知道它会根据看到的结果算出什么；甚至可以根据当前时间等外部信息再计算。
- 算完以后再 `UPDATE`。
- 更新时可能冲突，可能死锁；可能是数据依赖，也可能是“我要锁你的对象、你要锁我的对象”的互相锁定。

- 若形成循环依赖，数据库检测到后会说这个事务不能继续，要中止（abort，原声近似 BT）并回滚（rollback）。
- 并发冲突链（整理者串联：用 A/B 指代两个事务）：A SELECT → 计算 → UPDATE；B SELECT → 计算 → UPDATE；若 update 彼此读取/锁定对方对象 → 循环依赖/死锁 → DB 检测 → abort → rollback。

Git 像乐观并发控制：先假设两个 transaction 不会冲突

- 讲者认为 Git 模型有点像**乐观并发控制**（optimistic concurrency control；原声近似 mist optimistic）。
- 为什么乐观：执行时假设两个 transaction 不会冲突。
- 这个假设在数据库系统中大概率成立。
- 例子：教务系统。每个同学读的是自己的数据、写的是自己的数据；选课选的是我的课，查看课表查看的是我的课表。两个同学哪怕同时开始选课，也是读我的、写我的，所以全局冲突不太可能发生。
- Git 开发同理：假设我们已经事先开过会（今天早上开过会），我可以顺着这条路先做这个，另一个人做那个；合并时大概率不会产生冲突。
- 因此不管是 **merge** 还是 **rebase**，都能顺利完成。
- 转折：讲者最后说，但是这就是“古法编程时代”的……

小结： 讲者把 Git 视为乐观并发控制：快照要留，但历史主要服务于继续前进；冲突用数据库式事务检测与回滚处理，而乐观假设只在冲突大概率不发生时成立。

第 18 部分

仙 | -107老师

主 Agent 规划, 子 Agent 上锁开干

Agent 只干一件事：推进一件事，把屁股都擦干净

- 主 Agent 负责规划
- 子 Agent 负责上锁开干
 - Agents 完全可以直接上互斥锁，把活干了，解锁

把协调放到什么时候？

- Git/JuJutsu 像是“事后”的 Multi-Version Concurrency Control：冲突检测，类似数据库
- ✦ 互斥锁：修改前先约定谁能访问这份共享状态，把冲突变成等待

绿导师原谅你了 bilibili

核心链条：Agent 提速 → 频繁 abort/冲突 → 从乐观修冲突转向悲观上锁

1. 如果 **agent** 快 1000 倍，而主进程/主协调者（转录作“主威镇”，按上下文理解为协调者）一下子分出很多任务，就更可能频繁触发 **transaction abort**（转录作“transaction 的 bot”）、类似 abort（转录作“OLBOT”）。
2. 在 AI 时代，这种 abort 不是大问题：agent 出错了、出了 **conflict**，就修，而且能修好。
3. 但讲者追问：是不是在 agent 时代，可以换一种 **并发控制** 方式？
4. 他给出的方向是：从 **optimistic** 的“大家先往前跑，跑了再说”，转向更偏 **悲观并发控制** 的“先 declare、先上锁、再执行”。
5. 具体做法是在 **git repo** 或快照上按 **scope** 上锁；冲突者等前一个 release，几乎避免所有冲突。
6. 代价是：**互斥锁** 的本质是不并行、不并发，与多处理器并行根本矛盾。

Merge/rebase 本身能完成，问题是“古法编程时代”的冲突处理节奏

- 讲者先承认：不管是 **merge** 还是 **rebase**，都能顺利完成。
- 但他把这种模式称作“古法编程时代”的做法；这里他提到可用 **SQL** 作比方（未展开）。
- 讲者认为，旧流程默认冲突不会以极高频率发生，或者发生后可以慢慢修。

操作系统/数据库的锁语义：先声明、先上锁、后到者等待

- 讲者提醒：并发控制不只在数据库里，操作系统课也学；操作系统里做并发控制靠 **上锁**。
- 锁和数据库里的 **transaction** 有点不一样：锁是对资源 **lock**。
- 讲者上课常说：“一把大锁保平安”。
- 做实验时，要访问可能冲突的资源，就先把它锁住；一旦锁住，别人再想 lock，就必须等前一个 **release**。
- 因此建立的是“先 lock、后 lock”的先后关系：必须等前面 release，后来者才能 lock。
- 这不是 **optimistic** 的“大家先往前跑，跑了再说”。
- 悲观锁要求在 **使用资源之前 declare** 对该资源的使用，用上锁的方式。
- 讲者感慨：只有在操作系统课、数据库课真学会这些东西，回到软件工程里想，会发现这些概念又再做一次。

讲者的具体方案：主 agent 规划后，在 git repo/快照上按 scope 上锁

- 因为 agent 动作非常快，讲者认为应该用更偏向 **悲观并发控制** 的方式。

- 具体设想：
 1. 今天要做什么，或有一个 **主 agent** 先规划。
 2. 规划完后，立刻把项目的某一个部分锁上。
 3. 在 **git repo**（转录作“git ripple”）或一个 **快照** 上支持锁操作。
 4. 把 **scope** 说好：现在锁住这些范围。
 5. 要做一件事，需要这些范围，就先上锁；上锁后做事，做完 **commit**。
 6. 另一边也可以上锁；但如果他想干的事情被另一个人锁上了，就必须等前面的人释放，才能再上锁。
- 讲者判断：这样几乎可以避免所有冲突。

文件级锁例子：避免同写测试/配置，同时其他 agent 继续推进

- 例如在 **文件级** 上锁，就能避免两个 agent 同时写同一个测试，或往同一个配置里做更改。
- 又因为 agent 数量足够多、速度足够快，即使有 agent 等待锁，整体项目进度仍可以往前推：也许还有别的 agent 可以继续运行。
- 这类似操作系统里实现并发程序。

锁的代价：互斥锁排斥并行，与多处理器并行根本矛盾

- 讲者提醒：哪怕做到“一把大锁保平安”，或把锁拆细（转录作“锁拆拆桥”），**互斥锁** 的本质仍是 **不要并行、不要并发**。
 - 它和多处理器并行有根本矛盾。
 - 按讲者说法，这会把一个本来能并行/并发的程序“退回”（整理者串联：即退回串行化/失去并行）。
-

AI 时代可从 first principles 重做版本控制，OS/数据库仍值得学

讲者把这些知识关联起来看：在 AI 时代，可以重新设计一个版本控制系统，因此软件工程肯定要发生很大变化。

- 从 first principles 出发，要管理的是：快照、版本、change，还要管理飞快的 change。
- 讲者问“答案是什么”：Git（原文识别为“get it”）可能是一条线；如果“用一种像这样的方式来管理软件”，可能又是另外一条线。
- 因此，学操作系统、学数据库仍然有用。虽然会痛苦、会怀疑：AI 都会写数据库了，SQL 查询（原文先识别为“C口查询”）早就不自己写了；但讲者明确说：答案是有用的，还是要好好学习。
- 他举数据库里的 advisory lock（原文识别为“adversary lock”）机制为例。这也是他喜欢上课的原因之一：如果不来上操作系统课、不上生成式软件工程课，可能就不会让自己在备课中把这些事在脑子里再过一遍，去问“是不是地球上所有人都错了”。
- 讲者解释，自己讲给你们时必须共情听众：听众只能接受比较平实、朴实的逻辑，不能直接说“操作系统是怎么设计的”；所以必须把最顶层的 idea 提炼出来，再试图让大家共情。他自评可能做得不够好，学生可以在 AI 帮助下重新理解。

版本控制只是工具，真正难的是进度和团队管理

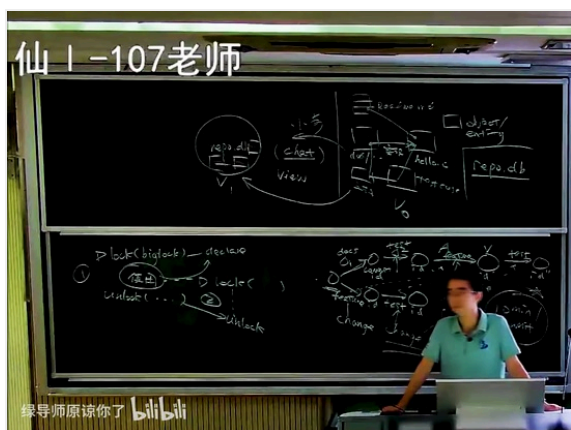
- 讲者对版本管理的讲述基本收束：从历史动机，到过去 Git，讲到现在，再讲到未来可能发生的事。
- 但他强调：版本控制本质上还是一个工具；哪怕是他刚才说的、基于乐观锁或悲观锁的版本控制，也一样。
- 真正困难的是进度，也就是团队的管理。
- 为什么现在还需要软件工程？因为现在还没有办法做到“一句话就直出一个高质量的产品”。

如果 AI 一句话做出微信级产品，软件工程就死了

- 讲者举例：如果“斩杀线”到了，老板有一个想法，比如“我要做一个 agent 用的微信”/“AI 时代的微信”，随口一说要做成社交爆款，以后大家社交不再用微信、而用“我”的；你可能还有更具体的思考。

- 此时如果 AI 直接就能做出微信级的产品，那么软件工程就已经死了。
- 讲者的判断链很直接：软件工程存在的理由是当前还不能一句话产出高质量产品 → 一旦 AI 能一句话产出微信级产品 → 软件工程的必要性就被打掉。

第 20 部分



团队协作

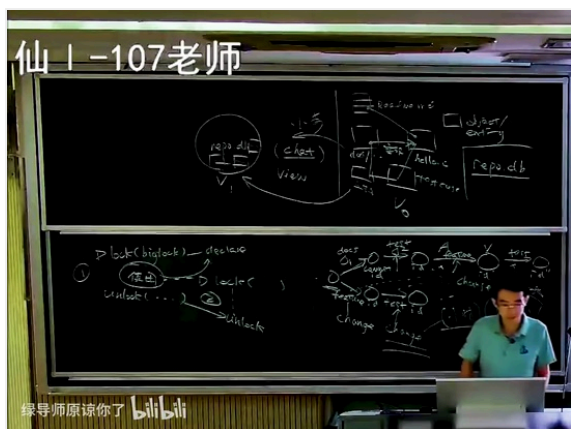
版本控制只是工具

真正的困难还是在**进度 (团队) 管理**。

AI 是一条“斩杀线”

- 如果一句话就能直出高质量产品，软件工程就直接**死了**
- Moore's Law (也是一种 Scaling Law) 还成立，模型就有机会更强、更小、更快
- 如果一个岗位 100% 的工作都能被 AI 替代，这个人在这个岗位上的价值在哪里？
 - 从某种意义上，被斩杀的数学工作者更惨 😞

1/5



AI 若能直接做出微信级产品，软件工程才“死”；现在斩杀线还没到团队管理

- 讲者设想一种更具体的情形：如果你有一些思考，而这时 AI 直接就能出微信级的产品，那软件工程就已经死了，大家也就“都死了”。这是极端化、玩笑式的说法。
- 但讲者判断：现在这条斩杀线暂时还没有到，至少没有到**团队管理**这一层面，所以还要学。
- 讲者进一步说：也许有一天自己真的也要被“拆掉”。理由包括：摩尔定律本质上也是 scaling law (原文近似 skin log)，现在仍然成立；模型肯定会更强、更小、更快；算力有一天也许能“唤出我们的智能”。
- 但今天至少 AI 还不知道怎么做团队管理。

团队管理难替代：多一个人，沟通关系平方级上升

- 讲者给出的核心机制：每当你多一个人，沟通关系都是平方级上升。
- 因此，软件工程团队里最难招的其实不是码农，而是那个能把任务分解的 **team leader**。
- 具体例子：要造一个“微信”这种**国民级应用**，一个 100 人团队能分成十个小团队，然后由十个 team leader 各自带人干，最后加起来还能拼成一个团队。讲者认为这种人最稀缺。
- 其次，能够带领小团队的人也很稀缺。
- 讲者引用《人月神话》：人与人两两之间就是平方级的沟通关系。
- 后果是：就算你能把自己份内的事情做好，因为平方级沟通关系，别人也在做他的事；你不百分之百知道别人在做什么，或者对整体团队了解不够好，最后就会各种冲突，比如“你写的测试我过不了”。

要练团队管理，就把自己当主 agent，把版本控制概念用到 agent 协作里

- 如果想管一个团队，或者毕业时具备一定团队管理能力，讲者建议：现在就可以把自己当成主 **agent**，像他刚才那样。
- 讲者承认自己刚才给了一个**不好的示范**：他说今天创了两个 **slave**（从属 agent，原文如此）。
- 你现在也可以立即拥有 **desk v4** 的 **slave**（原文如此，疑为某类 agent 产品或版本及其从属 agent）。
- 然后去思考：如果要尽可能最大化两个、三个或更多 agent，应该怎样**分解任务**；应该怎样确定**提交策略**；什么时候 agent 应该**提交**；什么时候应该**合并**。
- 这样就能把前面讲过的所有**版本控制概念**带到自己的开发流程里。

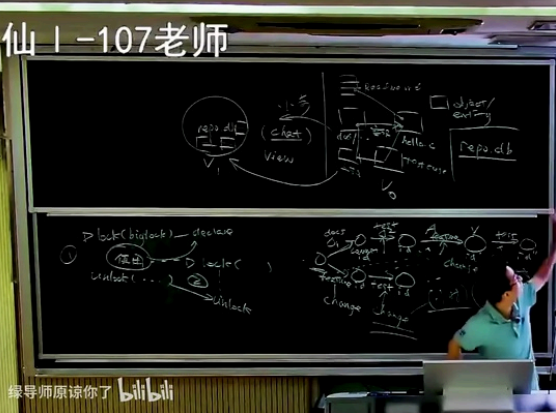
版本控制是经验蒸馏出的工具；在它之上长出自己的软件工程直觉

- 讲者强调：**版本控制是工具**，它蒸馏了人类过去很多年软件工程的经验，得到的是“人类要这样开发”。
- 在这个工具上，你可以开发出属于你自己的**软件工程的直觉和经验**。
- 他提到“学东西还是有用的”，并说后面可能再讲怎么做软件管理。

小结：AI 尚未越过团队管理这条斩杀线；团队管理的瓶颈是平方级沟通，因此最稀缺的是能分解任务的 team leader；版本控制是把人类软件工程经验带入 agent 协作的工具。

第 21 部分

仙 I-107老师



绿导师原谅你了 bilibili

再次回到 Pico Playground

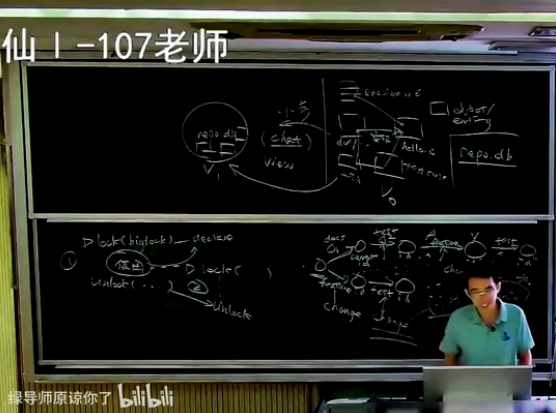
小学生的 Tower of Hanoi

- 考了个不容易的题目，一片哀嚎
- 如果为了挣 YYY，我们当然应该提供**增值服务**
 - 在服务器端保存小朋友的做题记录

GPT：数据库启动

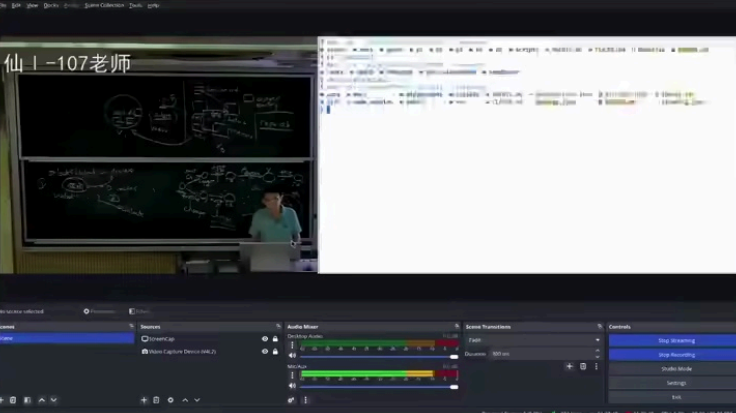
“ 一个 API 应用、一个关系数据库、一个对象存储；需要异步点评时再加后台任务进程。数据库存关系和记录，大课程包与完整轨迹放对象存储。任务量小时，数据库任务表就够用。”

仙 I-107老师

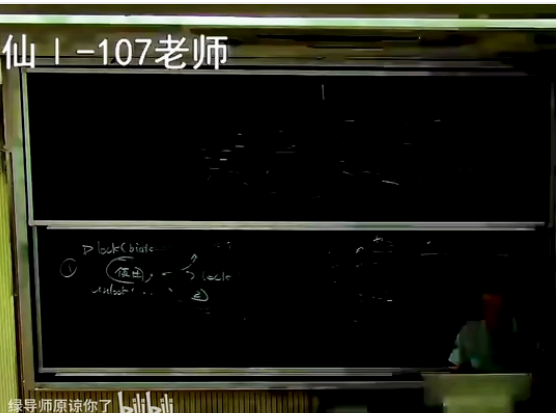


绿导师原谅你了 bilibili

仙 I-107老师



仙 I-107老师



绿导师原谅你了 bilibili

```
$ cd pico-playground/
$ eza --git --group-directories-first --icons=auto
.  apps  docs  playgrounds  scripts  AGENTS.md  package-lock.json  picopgUI.html  tokens.css
  dist  node_modules  public  src  CLAUDE.md  package.json  README.md  tsconfig.json
$ npm run dev
npm error Missing script: "dev"
npm error
npm error To see a list of scripts, run:
npm error   npm run
npm error A complete log of this run can be found in: /home/.npm/_logs/2026-09-15T11_33_16_831Z-debug-0.log
$ npm run dev

> learn-python@1.0.0 dev
> node scripts/dev.mjs

构建完成: dist/24point-ds.html (29.7 KB)
构建完成: dist/24point-gpt.html (18.6 KB)
构建完成: dist/chickenrabbit.html (24.6 KB)
构建完成: dist/cow-grass.html (33.0 KB)
构建完成: dist/hanoi.html (12.5 KB)
构建完成: dist/hello.html (4.6 KB)
构建完成: dist/picoworld.html (4.0 KB)
dev server: http://localhost:5173/
24point-ds.html http://localhost:5173/dist/24point-ds.html
24point-gpt.html http://localhost:5173/dist/24point-gpt.html
chickenrabbit.html http://localhost:5173/dist/chickenrabbit.html
cow-grass.html http://localhost:5173/dist/cow-grass.html
hanoi.html http://localhost:5173/dist/hanoi.html
hello.html http://localhost:5173/dist/hello.html
picoworld.html http://localhost:5173/dist/picoworld.html
```

单向限制让汉诺塔的常规递归方案失效

讲者最近看到一道题：在汉诺塔上增加额外要求——所有盘子的移动只能沿同一个方向，不能折返。讲者说它常被当作面试题，也像小学生考级题；常规汉诺塔大家会递归，但这道题不行。讲者还带了一个模型/道具，并准备了一个小程序演示。

- 两个盘子时还不明显。
- 三个盘子时问题暴露：常规汉诺塔会先把上面两个盘子移到中间，再把最大盘从当前位置移到目标；但在单向限制下，最大盘不能这样直接移过去，只能沿规定方向走。

最优解的关键：最大盘被单向约束逼成“向右两次”，小盘递归让路

讲者把问题拆开看最大盘：

1. 在受限规则下，最优解里最大盘不能一次到位，而总是先向右移一个，再向右移一个。
2. 为了让最大盘完成这两步，它上面的那一摞盘子必须先被移开、让路；最大盘右移一次后，小盘还要再调整，之后最大盘再右移一次，最后小盘再归位到目标。
3. 因此处理小盘的部分仍是递归/归纳：先移开、再移回、再移开，围绕最大盘的两步移动展开。
4. 讲者称，这样最大盘的移动次数最少，是不可避免的；剩下的部分基于 **induction/归纳法**，同样是最优。

逻辑链：

常规汉诺塔：小盘让路 → 最大盘一次到目标

加单向限制 → 最大盘不能一次到目标

→ 最大盘必须沿规定方向分两次移动

→ 小盘必须分三段递归让路/归位

→ 最大盘移动次数不可再省，剩余由归纳法保证最优。

程序演示证明的是“最优”，不是“能解”

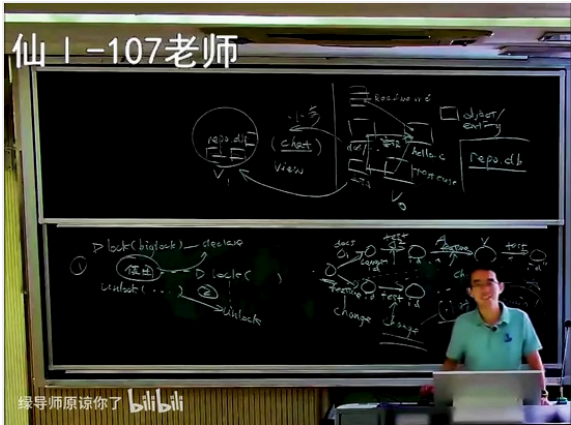
- 讲者写了一个程序，按上述思路处理：如果最大盘还没到目标位置，它总是向右移一个、再向右移一个。
- 讲者认为，这使最大盘移动最少，无法再优化；剩余部分由归纳法也是最优。
- 讲者感叹这题有点难，很难想象小学生能做。
- 他调侃如果放在**保研机试**里，要求输出方案或输出数字， n 也不大，但确实需要先理解问题。

汉诺塔不是重点：重点是把产品卖给小学生，靠付费增值服务

讲者明确说，他不是再讲汉诺塔的故事。重点是“现在要推出这个产品”：例如把这个东西卖给小学生。问题不是算法怎么最优，而是怎么最好地卖给小学生。讲者给出的直接答案是：提供付费的增值服务。

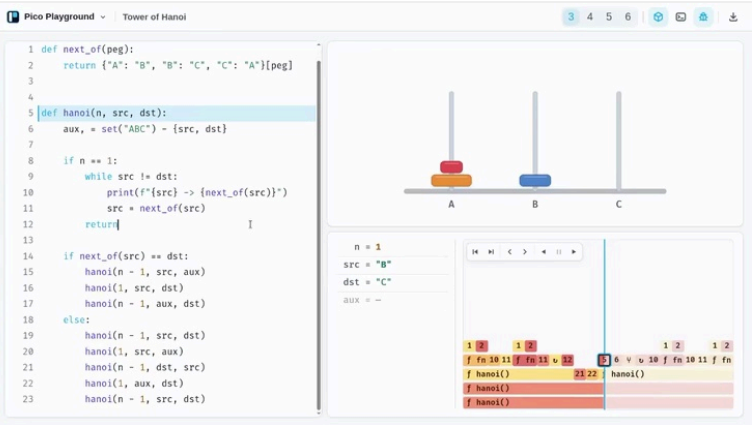
小结：讲者先用受限汉诺塔说明问题理解与最优归纳，随后把话题拉回产品管理——面向小学生销售时，他主张用付费增值服务。

第 22 部分



仙一-107老师

绿导师原谅你了 bilibili



Pico Playground - Tower of Hanoi

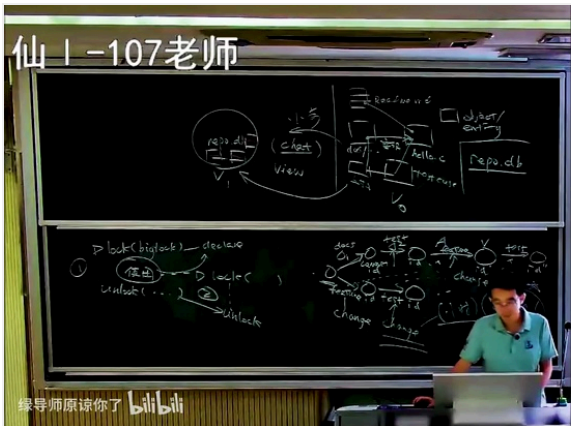
```
1 def next_of(peg):
2     return {"A": "B", "B": "C", "C": "A"}[peg]
3
4
5 def hanoi(n, src, dst):
6     aux = set("ABC") - {src, dst}
7
8     if n == 1:
9         while src != dst:
10             print(f"{src} -> {next_of(src)}")
11             src = next_of(src)
12         return
13
14     if next_of(src) == dst:
15         hanoi(n - 1, src, aux)
16         hanoi(1, src, dst)
17         hanoi(n - 1, aux, dst)
18     else:
19         hanoi(n - 1, src, dst)
20         hanoi(1, src, aux)
21         hanoi(n - 1, dst, src)
22         hanoi(1, aux, dst)
23         hanoi(n - 1, src, dst)
```

n = 3
src = "B"
dst = "C"
aux = -

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23



仙一-107老师

绿导师原谅你了 bilibili

再次回到 Pico Playground

小学生的 Tower of Hanoi

- 考了个不容易的题目，一片哀嚎
- 如果为了挣 ¥¥¥，我们当然应该提供**增值服务**
 - 在服务器端保存小朋友的做题记录

GPT：数据库启动

“一个 API 应用、一个关系数据库、一个对象存储：需要异步点评时再加后台任务进程。数据库存关系和记录，大课程包与完整轨迹放对象存储。任务量小时，数据库任务表就够用。”

4/5

能收费的不是 playground，而是“独门 prompt + 个性化 AI 助教”

- 讲者设想把产品卖给小学生。现有形态是一个 playground：小学生可写代码，代码可随时编辑；可视化随代码运动；写错时能定位到具体语句，让学生找到 bug。例子：试图把蓝色移到小的/大的上，出错后能找到那条语句。

- 要赚钱，讲者认为最好提供**付费增值服务**，例如 **AI 助教**。
- 变现机制：小朋友卡住时点“我要助教”，后台开始消耗 **DeepSeek** token（原文 ASR 作“deep pick”）；讲者收取比 DeepSeek 原价更贵的 token 费，并记在账户上。
- 为什么敢收费：讲者认为自己的 **prompt 是独门的**；作为有经验的老教师，可以把自己“蒸馏”成 prompt，敏锐知道小朋友的问题在哪里，给出个性化指导。
- 讲者补了一句“这个是不是不太好啊，开玩笑的”，随后说还是尽可能以**便宜成本**提供给小朋友。

要做 AI 助教，就绕到信息系统；GPT 给出的是一套标准 Web 后端

- 讲者没时间，让学生做；学生和 AI 聊完一版设计后，又让 **GPT** 梳理需求。
- GPT 原话/大意：一个 **API 应用**、一个**关系数据库**、一个**对象存储**；需要异步点评时再加后台任务进程；数据库存关系和记录；大课程包与完整轨迹存放对象存储；任务量小时，数据库任务表就够用。
- 讲者评价：如果是软件工程课，这“太正常了”；但他不喜欢这个设计。

不喜欢的原因：Overleaf 的 Git 导出 + Codex 本地改论文

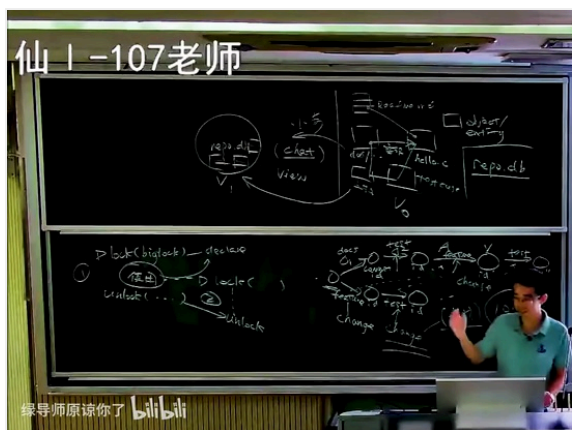
- 讲者拿 **Overleaf** 协作经验作对照：Overleaf 可以把文档当 **Git repo** 导出。
- 他最喜欢这点，因为要给学生改论文时，可以：
 1. 启动 agent；
 2. 把论文**克隆到本地**；
 3. 不必忍受网页界面；
 4. 打开 **Codex**；
 5. 让 Codex 直接给每个 **.tex** 文件修复语法错误；
 6. 给一段很长的 prompt，让它遵循要求，论文就改好了。
- 这里他实际看重的是：**文件能导出、能 clone 到本地、能交给 agent / Codex 批处理**，而不是被锁在网页界面里。
- **（整理者串联）** 讲者没有把“不喜欢 GPT 设计”的理由完整说穿；从 Overleaf 例子看，他偏好的系统可能不是纯 API+关系数据库+对象存储的 Web 服务，而是让课程包、论文、轨迹等文件可克隆、可 agent 操作的工作流。

逻辑链

1. 已有 playground: 写代码 → 可视化跟随 → 报错能定位到语句。
2. 要赚钱 → 加付费 AI 助教。
3. 敢收费 → 独门 prompt + 老教师经验蒸馏 → 个性化指导。
4. 要实现助教 → 必须做信息系统。
5. 学生/GPT 给出标准后端需求: API、关系数据库、对象存储、异步任务。
6. 讲者不喜欢 → 用 Overleaf/Git/Codex 改论文的经验, 说明他重视本地、可克隆、可 agent 批处理的流程。
7. **(整理者串联)** 因此他想要的信息系统, 可能与 GPT 那版标准设计不同, 应围绕可被 agent 操作的文件/轨迹来组织。

小结：这段从“AI 助教怎么收费”转到“为什么讲者不接受 GPT 的标准 Web 后端设计，而偏爱 Overleaf 式可导出 Git、可本地 agent 处理的流程”。

第 23 部分



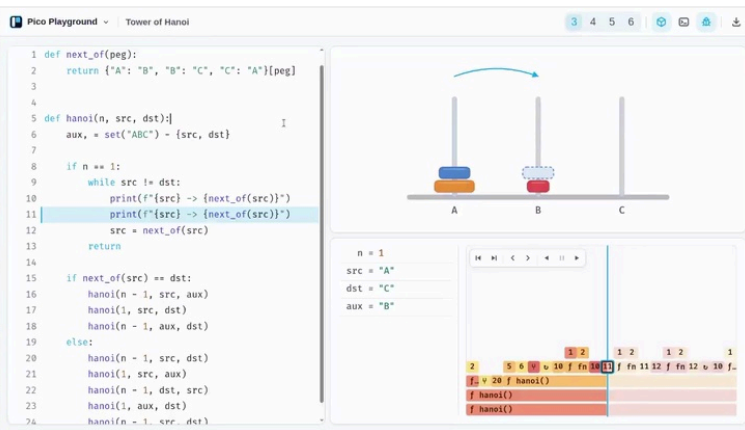
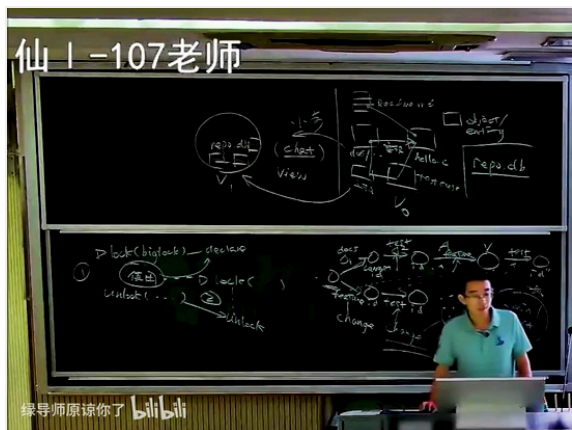
但 Git 不刚好适合这个吗？

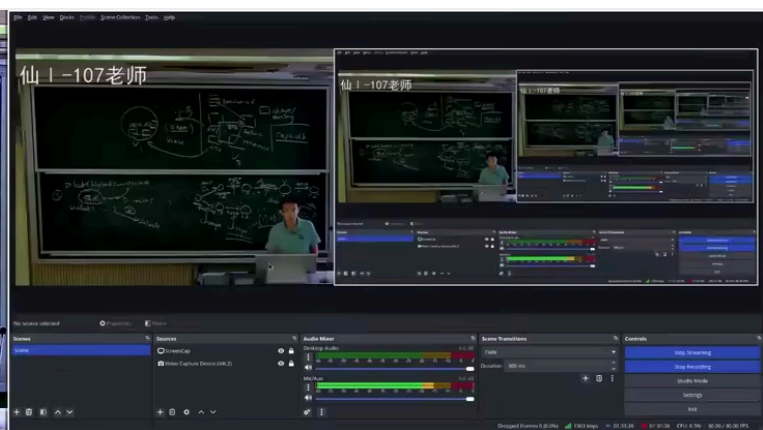
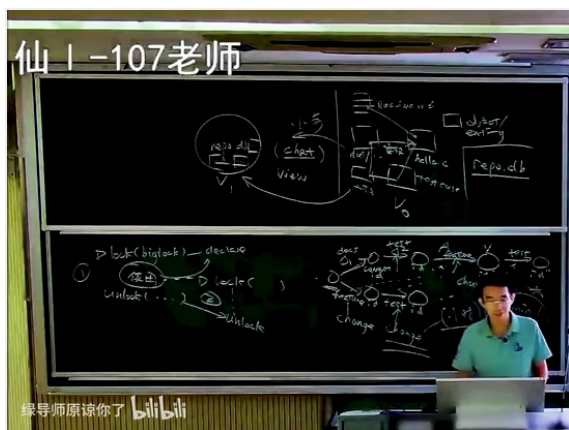
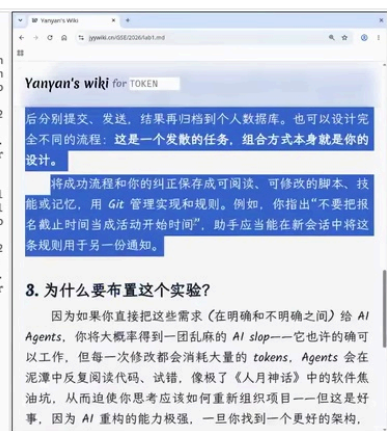
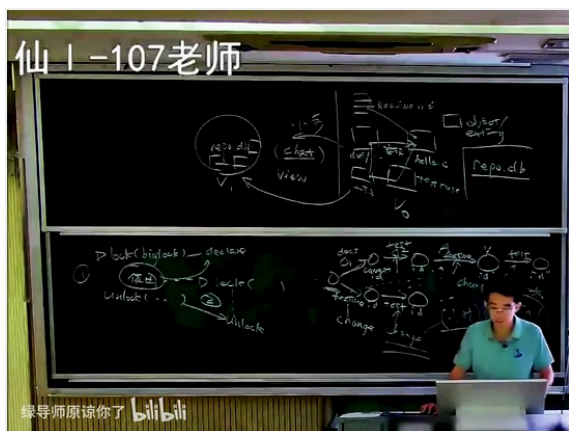
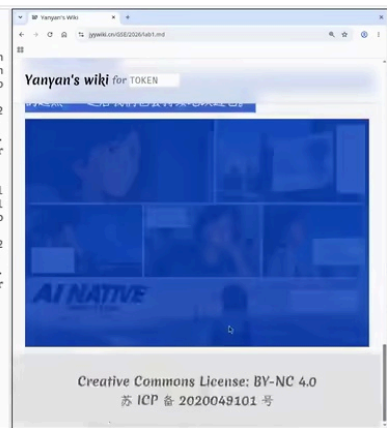
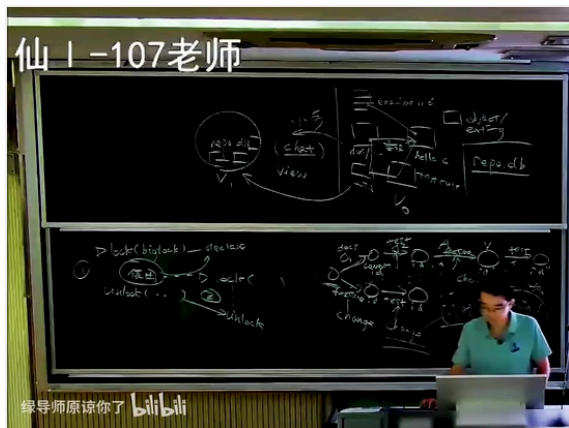
GPT 没想到

- **树状版本历史:** ➡ 做到哪一步、从哪里重试, 都可以留下来
- **免费的 Diff 功能:** ➡ 前后两次提交, 到底改了什么?
- **Comment Commit:** ➡ 把点评关联到当时的版本: `git notes` 可以后补点评, 不改原提交

记录“怎么做出来的”

- ◆ 汉诺塔只保存最终盘面，看不出孩子在哪里卡住：保存动作序列，才能回放、比较和点评。
- ◆ 把动作保存成逐行文本，Git 就能提供版本与差异：“哪一步值得表扬”，仍然要由应用理解。





长 prompt 能把语法错误改到“质量论文”标准，真正要紧的是后面的版本管理设计

- 讲者说：里面的语法错误，可以给一个很长的 prompt，让 AI 遵循“质量论文”的标准去改；这个改法已经改好。
- 接着转到 pickle playground (ASR 原词；平台名不确定) 的设计。

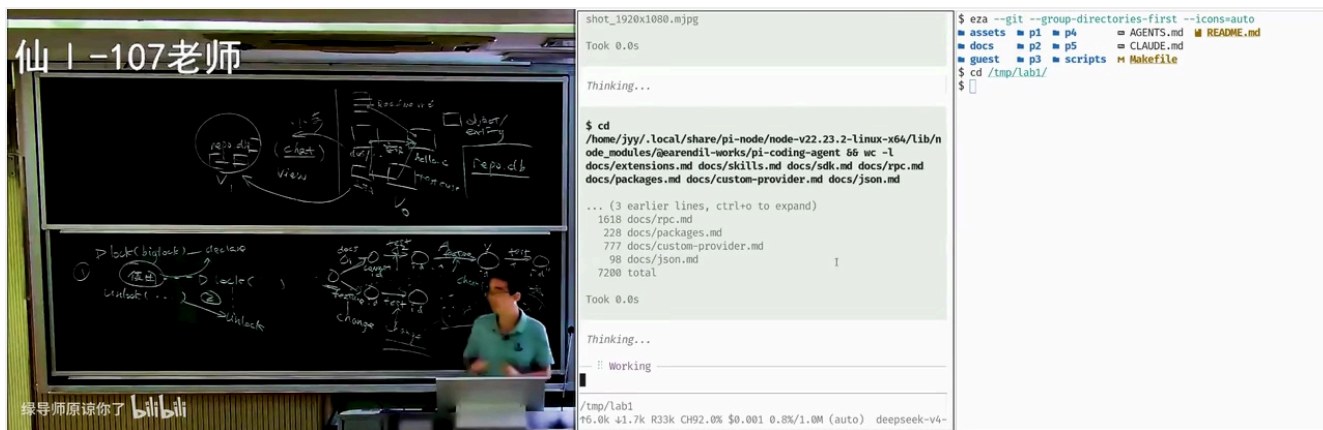
自动 git 提交比数据库 persist 更贴合“版本管理”这个本质需求

- 设想：学生每改一次——例如 **idle** (ASR 原词为“idol”)、点运行、切换——就自动做一个 **git commit**。
- 当申请 **AI 助教** 时 (AI 助教转圈圈)，可以在 **git commit** 上产生一个分叉，作为一个 **comment**。
- 讲者认为 **GPT** 没有算到：产生这个 **commit/comment** 的成本，相对于把剩下的东西 **persist** 下来，可以忽略不计。转录此处出现“远远大于……成本是忽略不计的”，字面矛盾；按上下文应理解为“远小于/可忽略不计” (**整理者按上下文修正 ASR**)。
- 逻辑链：
 1. 自动 **commit/comment** 成本很低 (讲者原意，见上)。
 2. 数据库确实有好处：总体压缩可能更小、性能更高。
 3. 但这些不是讲者要的；他的本质需求是 **版本管理**。
 4. 所以早期可以把代码库和这个东西连在一起，把学生 (讲者说“小朋友”) 的代码直接存在代码库里，在上面创建探索性的 **git 历史记录**。
 5. 直到文件目录格式、**commit message** 格式、如何使用这些文件都收敛，再迁移到数据库。
- 讲者认为这都可以，挺好玩。由此他说：学任何东西都有用；学 **git** 有用；学 **multi-agent** (ASR 为 **mod agent**) 管理，就有机会想到一个和别人不太一样的设计。

项目早期不要用 multi-agent：先当包工头，用最强 agent 多轮对话

- 讲者断言：项目早期肯定不能用 **multi-agent**。
 - 早期你就是 **包工头**；应先找一个相对能力强、你能用到的最强 **agent**，跟它多轮对话。
 - 他鼓励：现在开始干比什么都好。
 - 他决定现场直接打开课程作业，完全没有事先准备；还说“开两个” (窗口/会话，转录不清)。两种办法：第一种是直接回车。
 - 讲者认为每个同学都可以体验；今天他替大家体验。更好的方式是观察它，看它会做什么。
 - 原因：有时 **AI** 的直觉和人的直觉不太一样。例子：一线模型会进入 **plan mode**，先跟你确认一些事实，再生成一个 **workflow**。
 - 另一个例子：**Claude Code** (ASR 为 **cloud code**) 在需要执行比较复杂任务时，会生成一段 **JavaScript**。
-

第 24 部分



Claude Code 用 TypeScript 里的自然语言 agent 函数组织 workflow

- 讲者举例：**Claude Code**（转录作“cloud code”）执行复杂任务时，会生成一段 **JavaScript/TypeScript**。
- 这段 TypeScript 里有一个函数叫 **agent**，里面是**自然语言**。
- 用这种方式组织 **workflow** 时，甚至可以有循环：有很多任务时，在循环上解决 workflow。
- 效果是它直接开干。讲者调侃“用魔法打败魔法”，并说 **300 token per second**（原音：“300 token persd”）太恐怖。

最危险点：prompt 的每个词都被 self-attention 看到，模型又成了“prompt 的舔狗”

- 讲者把这一点称为上下文工程/提示词工程里最危险的地方。
- 逻辑链：
 1. prompt 里每一句、每一个词，**self-attention** 都会看到。
 2. 看到后，因为 AI 现在被训练成 **prompt 的舔狗**，它有一种无法往外拉的倾向：必须遵循你的指令。
 3. 所以当需求——尤其是开放需求、讨论稿式需求——丢给 AI 时，它会**过度遵循指令**。
 4. 最后生成的东西确实满足指令，**但不是你想要的**。

实验要求的真正重点是言外之意：“实现一个持续为你工作的个人助手”

- 讲者回头看实验要求：字面上有 1、2、3、4、5 等解读，但真正最重要的是那句话：实现一个持续为你工作的个人助手。
- 其他话相对不重要，只是对“好助手”的解读。
- 讲者问听众：如果要实现个人助手，第一个 feature 是什么？一个月内最大的痛点是什么？
- 答案是**高效学习和复习**；因为要卷成绩，所以这毫无疑问是最重要的学习场景/需求（原音作“学习排版案”，词不清）。
- 理想学习助手：自动从外网收集数据，层次化归档，并给一个好界面。讲者说这是你最想要的。

AI 没做学习助手，而是掉进 database/通用功能，暴露 intent 与输出的巨大 gap

- 讲者估计 AI 十有八九没做上面那个。
- 他问 AI：想要一个比较好的学习助手，能收集外部信息、综合总结、根据我的进度学习，看哪部分能实现。
- 看到 AI 在做 **database**。
- 讲者总结：在没有你精心设想过、讨论过的时候，AI 基本上只会生成某些通用模式（原音含“slot out”“SP2”“S mail”等，具体术语无法可靠还原）；它没做到你真正要的东西。
- 结论：你脑袋里所想的 **intent**、**specification**、**intention**，与 AI 输出之间有巨大 gap。

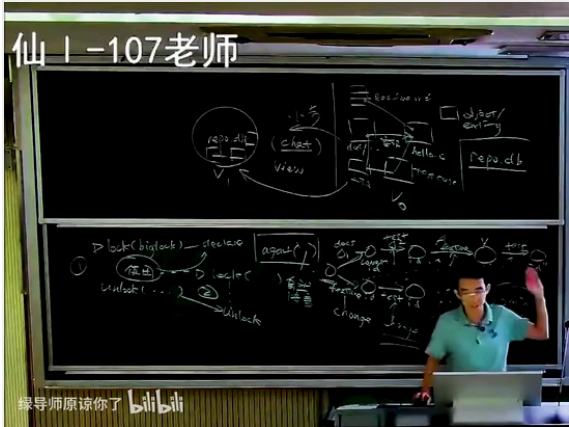
一旦模型看到目标，它就真的去实现；但可能实现错目标

- 讲者说：一旦模型看到“你要实现这个”，它就真的去实现它。
- 但他觉得它肯定做了一些不好的事情。
- 例子：虽然当他提“一号”时，他真正需要的是一个 **Browser Use agent**。
- （整理者串联）所以问题不是模型不执行，而是它把被写出的字面指令当成目标；越是开放需求，真实 **intent** 越容易漏掉。

小结：AI 会高效执行你写出的字面指令；没有写出的 intent/spec 才是 gap 所在。

第 25 部分

仙 I -107老师



绿导师原谅你了 bilibili

```
console.log("\n== drafts ==");
for (const d of listDrafts(db)) console.log(d.id, d.status,
d.subject, "\n" + d.body.split("\n").slice(0,6).join("\n"));
console.log("\n== rerun idempotency ==");
const again = await runNoticeWorkflow(db, cfg.pdbDir, ehall,
{});
console.log("second run outcomes:", again.length,
"confirmations:", listConfirmations(db).length);
db.close();
EOF
cp /tmp/t_flow.ts ./t_flow.ts 66 rm -rf state-flow 66 node
./t_flow.ts 2>&1 | grep -v ExperimentalWarning | grep -v
trace-warnings; rm -f t_flow.ts

... (38 earlier lines, ctrl+o to expand)
我会在截止日期前按要求完成提交。谢谢您的提醒。

=== rerun idempotency ===
second run outcomes: 0 confirmations: 2

Took 0.1s

Thinking...

F Working

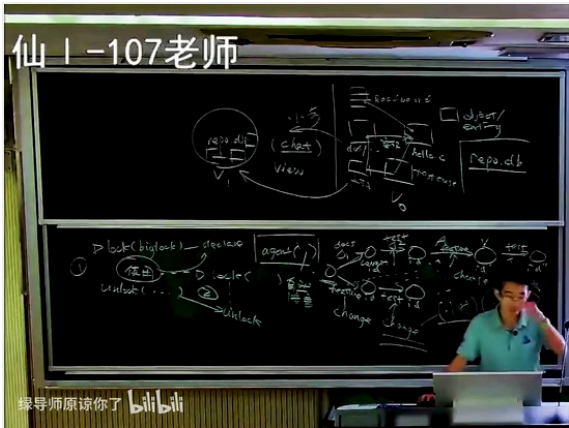
/tmp/lab1
#33k 468k R3.3M CH99.7% $0.033 9.1%/1.0M (auto) deepseek-v4-f
```

你要的三件事：收集外部信息 → 综合总结 → 根据我的进度学习。

你的需求	项目里最接近的部分	够不够
收集外部信息	small/(transport 抽象 + 写等逻辑 + threading) 是最完整的“外部源接入范式”；ehall/http-transp-ort.ts 是抓网页的样板	范式有了，但只接了邮箱，没有 web/RSS/arXiv 源
综合总结	small/reply.ts 的 composeReplyPrompt()：检索 pdb → 拼上下文 → 交给 LLM，还自带引用格式和确定性兜底 templateReply()	框架对了，但没有“大脑”去调它；daemon 不存在
根据我的进度学习	✗ 没有。pdb 只是你手写 md 的只读索引；tasks/confirmations 是 workflow 状态，不是学习状态	完全没有“知识点/掌握度/复习”数据模型

一句话：pdb (贴资料) + small (接外部源) + reply.ts 的“检索上下文-生成”链路，这三块拼起来就是你想要的骨架—but 缺了 brain/daemon 这个执行体，也缺学习进度这一维。

仙 I -107老师



绿导师原谅你了 bilibili

开始做一个项目

你就是包工头

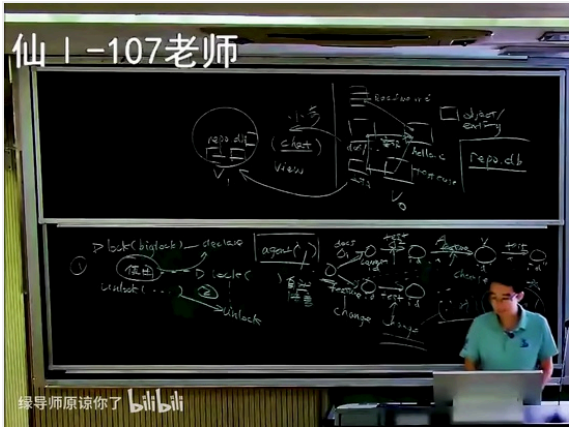
不如我们直接拿 Lab 开干吧！

- 任何时候，**开始都比不开强**

从零开始：连需求都还没对齐

- “从零开始做产品”的难度比“参与一个成熟开源项目”大 (intent-spec-impl gap)
 - 任何“确定”的东西，都可能是把对需求的**误解**固定下来
- 准则：没有什么是不变的，能不能以不变应万变？
 - 其实就是：你的软件能否抵抗**需求的变化**？

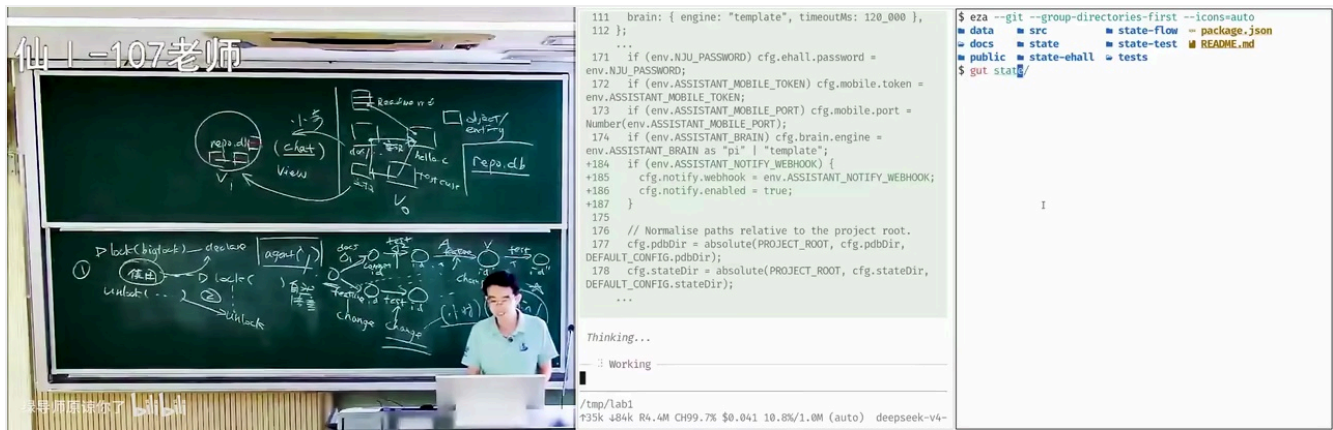
仙 I -107老师



绿导师原谅你了 bilibili

仙 I -107老师





看起来每步都正确，不等于得到想要的最终结果

- 讲者先说：这个 Agent 真的是去实现了；但讲者认为它“肯定做了一些不好的事情”。
- 具体例子：当他提出“一号”（转录中一处作“L1号”）时，他真正需要的是一个 **browser use agent**，能在各种网页上帮他处理事务。“一号”只是作为测试出口，能把他的个人信息导出到“一号”上面。
- 讲者由此说：第一步实现时，不需要在“教务系统”上做太多设计；作为过渡设计，设计得太多反而不好。
- 这和前面展示的例子非常相关：从 **README.md** 来看，它做的所有事情都是对的，没有一个错误；但最终得到的是你不想要的东西。
- 细节证据：它甚至没有创建一个 **Git repository**。讲者说，这就是“你不跟它讲”的结果。
- 讲者推测：如果是 **Codex**，它可能还是会初始化一个目录，或者会问你：这个项目应该怎么管理？
- 讲者对比成熟产品经理的做法：如果一个成熟产品经理接到老板交代的项目，他肯定会先和团队沟通项目怎么管、协作怎么做、每天什么时候开会——甚至发生在写下第一行代码之前。
- **Agent**（转录作“A震”）则很好玩：它直接上手就做，把它当成一个用一次就丢掉的东西。
- token 成本上，讲者说感觉它烧了不少 token；不过他参与 **DPC 哈里斯内测**（原文如此）获赠 1000 块额度，到现在还没完全用完，所以在刚开始时 token 成本也无所谓。

不能把任务直接丢给 Agent：先建空仓库，再小步走

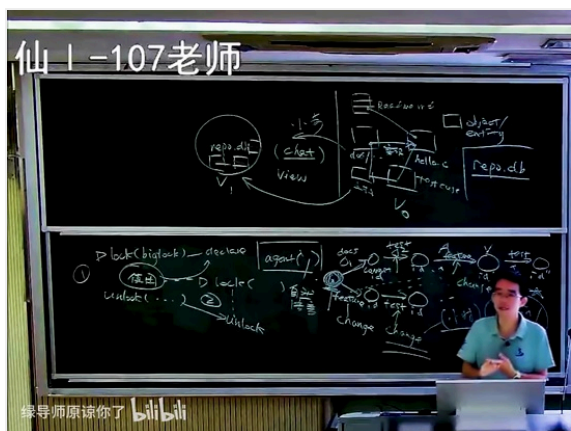
- 讲者结论：刚开始时，脑子里还是要回到项目、回到管理。不能直接把任务丢给 Agent。
- 他给出的顺序是：

1. 第一件事一定是先创建一个空的仓库/项目。
 2. 创建完之后，再想下一步要迈向哪里。
 3. 下一步不能大跨步，要往前迈一小步，再往前迈一小步。
- 讲者判断：现在 AI 从零开始做一个东西、做出 **slop**，比它到一个成熟项目里做维护性任务要难很多。

软件工程的真正考验：能否“以不变应万变”

- 讲者说，如果他要和一个 team 或一个学生谈论“要设计一个什么软件”，他的问题是：**你能不能以不变应万变？**
- 也就是说：现在设计的软件架构，或者流程方法，能不能抵抗未来需求发生根本性的变化？
- 例子：需求表面上可能是“生成一个挣钱的产品，去挣宝妈们的钱”；但讲者真正需要的是一个**能扩展的架构**。
- 未来会变化的东西包括：别的需求来了、AI 模型发生变化、软件工程/合作模式发生变化。
- **AI 助教**的例子：它可能是文字助教，也可能是图像助教；可能有交互，也可能没有交互；各种需求在不断变化。
- **（整理者串联）** 因此关键不只是让 Agent 完成眼前一步，而是让架构和流程在需求根本变化时仍然承受得住。

第 26 部分



开始做一个项目

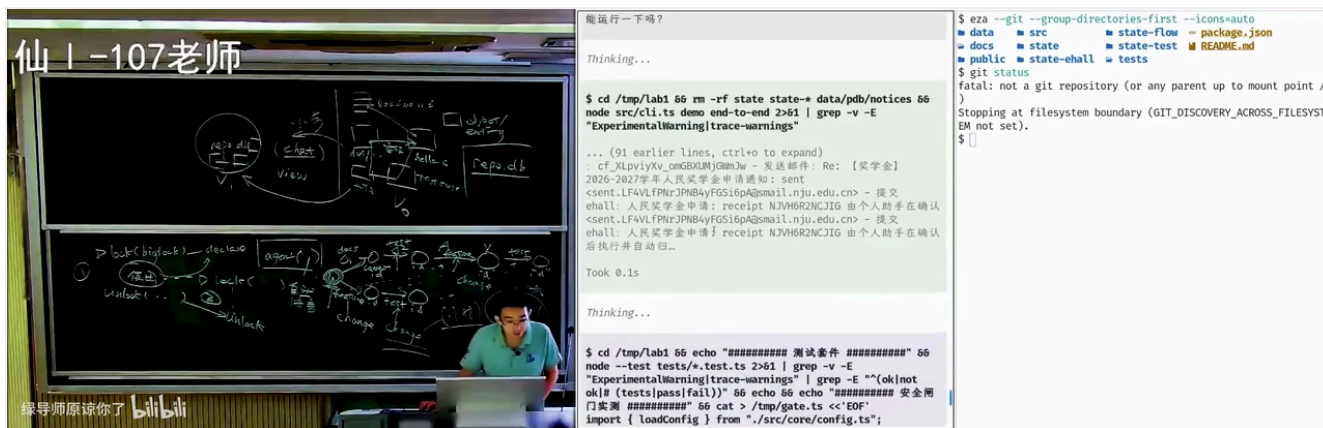
你就是包工头

不如我们直接拿 Lab 开干吧！

- 任何时候，**开始都比不开始强**

从零开始：连需求都还没对齐

- “从零开始做产品”的难度比“参与一个成熟开源项目”大 (intent-spec-impl gap)
 - 任何“确定”的东西，都可能是把对需求的**误解**固定下来
- 准则：没有什么是不能变的，能不能以不变应万变？
 - 其实就是：你的软件能否抵抗**需求的变化**？



需求会变，架构要用最小变化应对变化

- 讲者从 AI 助教 展开：AI 助教可能是文字助教，也可能是图像助教；可能有交互，也可能没有交互。
- 当这些需求不断变化时，他提出的核心问题是：我的架构还有没有可能以最少的变化去应对需求的变化。
- 讲者认为，这正是软件工程最不一样的地方：数据库、操作系统往往会解释“为什么今天的数据库是这样、为什么要并发控制”等原因；而软件工程要面对的是未来需求的变化。
- 逻辑链：
 1. AI 助教的需求形态多样且会变。
 2. 如果每次变化都大改架构，代价高。
 3. 所以架构应尽量用最小变化吸收需求变化。
 4. 讲者把“应对未来需求变化”放在软件工程的核心位置。

版本管理三件事：记住现在、记住历史、记住教训

- 讲者在讲版本管理时说，版本管理做三件事：
 1. 记住现在：必须有一个 **repository**。不管是某种形式，还是 C/C++、Python、JavaScript 项目，都要把文件管起来。
 2. 记住历史：某天某个东西不要了，或删掉后才发现有用，还可以把它捞回来；从提交历史中能看到自己如何演进到某一点，发现某件事失败，可能停在那里，再回到另一个点，开始另一条正确的路线。
 3. 记住教训：版本控制里的 lessons 不停警示你该怎么做，怎么从第一步开始迈出向后的一个小步。

- 讲者强调：这三件事在 **agent 时代** 和软件工程的“古法编程”时代都成立。
- 逻辑链：repository 管住现在 → 历史可恢复、可回溯 → 从历史中吸取教训 → 指导下一步演进。

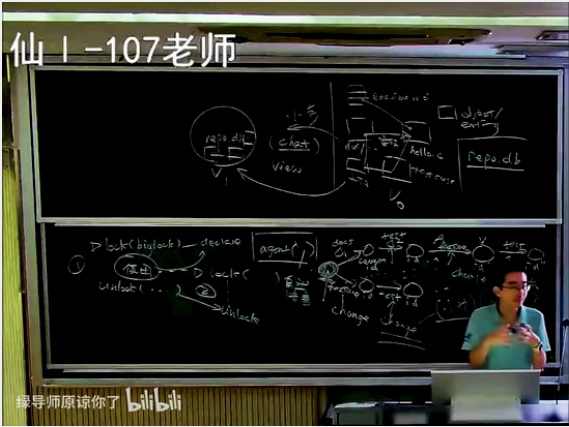
学生项目：真代码、真 HTTP 服务，但被做成业务逻辑系统有风险

- 讲者看一个提交/项目时提到：已经做了“提交人民奖学金申请、准备退课、看有没有 get”等事（功能名按转写保留）。
- 他的评价是：**引擎全部是真代码，手机端是真 HTTP 服务**，模拟部分/邮箱没有（此处 ASR 不清，保留原词，不展开）。
- 架构上有 **入口层、编排层、能力层**，讲者称其“非常非常典型”；**数据层**他说不想再 re-review。
- 但讲者指出风险：学生显然把它当成一个**业务逻辑系统**来做，这个软件已经看起来像一个**教务系统/教育系统**；他还说下节课会讲教务系统。
- 讲者认为这有很大风险：它容易把大量并非真正需要的东西先实现出来。

从零构建更符合 best practice 的做法：先做 MVP，逼清楚绝对要的需求

- 讲者提出，如果真要从零开始构建，现在更符合主流 best practice 的是用 **MVP**：**minimum viable product / minimal viable product**，也就是一个**最小验证版本**。
 - 他引用：The biggest risk in product development is building something nobody wants.
 - 讲者用学生项目说明：他做的东西就是“我不要的”；绝大部分东西其实不是我要的。讲者说自己在看他代码时已经解释过自己真正想要的是什么。
 - 因此，MVP 在讲者这里的意义是：**逼迫你想清楚，什么样的需求才是你绝对要的**。
 - 最后讲者转向一个未说完的例子：如果我要做一门课程的（助手），或者如果我要做一个个人助手……（话未完成）。
-

第 27 部分



用 MVP 验证需求

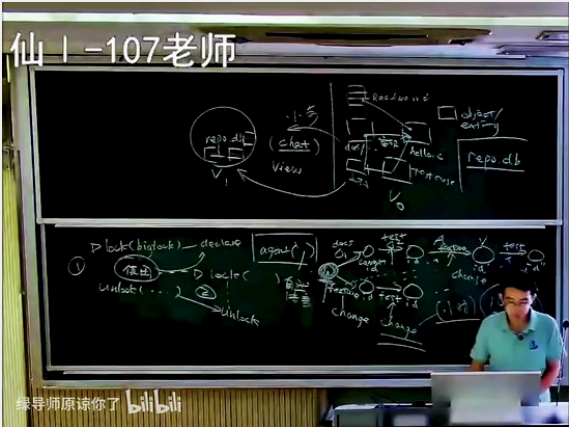
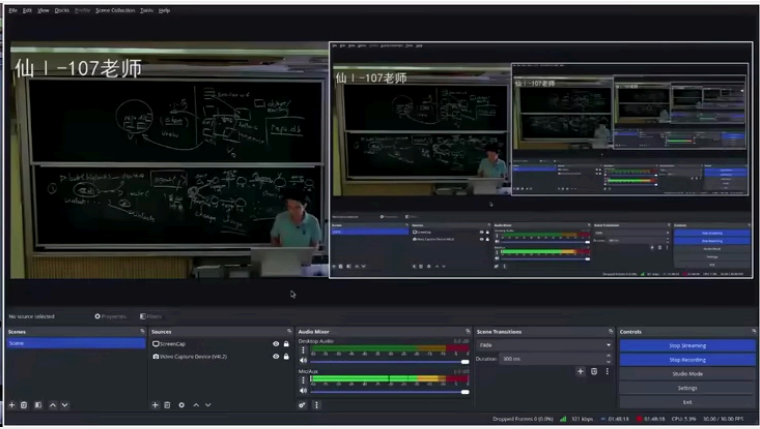
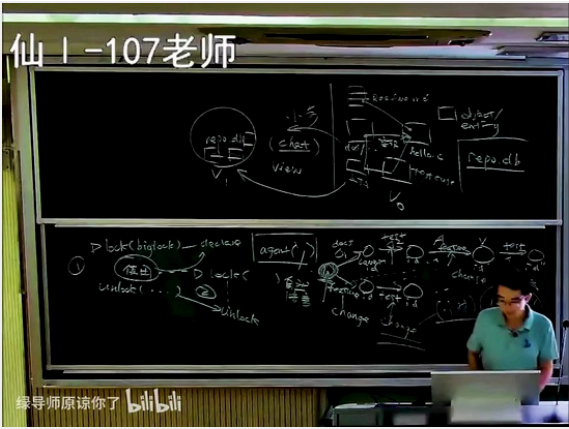
Minimum Viable Product: 用一个最小的版本验证

“ The biggest risk in product development is building something nobody wants.

- MVP 会给你一个**立即能使用、不完备**的版本
- 帮助你考虑未来潜在的变化、评估软件的设计

先验证什么？

- ✦ 先选一个最值得验证的假设，再做能验证它的最小版本。(Eric Ries: MVP)



unicode61 切成 token.

- 单一执行闸门：任何对外动作都是 executeConfirmation，状态机 pending → approved → executed 单向，markExecuted 带 WHERE status='approved'，并发/重试等。
- 发送前冻结：确认时把主题/收件人/正文（或表单字段）写进 confirmation payload，执行时读 payload 而非草稿——“发出去的就是你确认的那一版”。
- 幂等去重：邮件 Message-ID UNIQUE + uidvalidity/last_uid 增量；草稿 idempotency_key；流程 workflow_runs(message_id) 只处理一次。
- 风险分级：服务元数据 low/medium/high，标题命中 退课|撤销|注销|drop... 自动升级为 high；medium 输 确认，high 输 我确认执行。
- 无常驻对话：手机提交的任务落 tasks 表，daemon 原子领取执行，关掉手机照跑；PWA 用 Bearer token + SSE 推送。

组合流程 (combine/workflow.ts)

```
...
未处理通知 -打分→映射 ehall 服务→抽取材料清单→pdb 核对（有依据/待确认）
  ↳生成陈述+填 ehall 表单→confirmation A
  ↳生成回复草稿并冻结→confirmation B
  ↓手机上各输短信确认
执行→收据/邮件号
data/pdb/notices/YYYY-MM.md→重建索引→下次可被检索引用
...
I
Recap: 采用分层阅读法（入口→编辑→能力→核心→数据）梳理代码，用 docs/DESIGN.md 的架构图作骨架并对照源码的 schema, executor, worker 校验，最终画出覆盖模块依赖、单一确认闸门和组合流程的 ASCII 图。核心洞见是“Markdown 真源 + SQLite 状态 + 单一执行闸门”这三条主线支撑起全部互项能力。
```

tmp/lab1 (master)
*6.7% 42.1k R26k CH91.6% \$0.002 1.2%/1.0M (auto)

deepseek-v4-flash • high

绝对要的需求：个人助手先要提升学习效率和工作效率

讲者先问：什么样的需求才是你绝对要的？以个人助手为例，讲者最想要的是两点：提升学习效率、提升工作效率。讲者认为，总结出这两条后，第一个 MVP 里可以没有任何数据库。

逻辑链：

1. 个人助手的核心需求是提升学习/工作效率。
2. 做 MVP 时，不必先上数据库。
3. 因为文件系统膨胀时，可以直接跟 Agent 说：按这个数据组织帮我提炼出一个数据库，问题就结束了。
4. 一旦一开始带上数据库，每次数据库重构都需要让 Agent 做 **database 迁移**。
5. 文件系统不同：你可以自己在文件管理器里调整目录结构，逐渐梳理到舒服的结构。
6. 调文件与调 **database schema** 的心智负担不一样。
7. 人类从小（小学电脑课）就操作文件目录，对直观模式非常习惯；很多同学还没学过数据库课，面对 database schema 是未知物，需要强行做一次概念翻译：“我脑子里是这种概念，要翻译到数据库里是那样。”
8. 所以讲者主张：不如用一个已经适应、熟悉的 system，用最简单的方式帮你管理学习资料、录入学习资料和通知消息等。

回邮件 Agent 实例：云盘 + 多级流水线，没有数据库

讲者说自己回邮件的 Agent 都没有用任何数据库。流程大致是：

- 我的 Agent 会收我的邮箱；
- 然后把 **email id** 放到一个共享云盘同步上；
- 这个 email id 里有 Agent 解析出来的内容，比如：我回复的草稿、邮件分类、是不是垃圾邮件；
- 再由下一级流水线去处理。

讲者强调，先要明白：

- 系统里有哪几级流水线；
- 这些流水线之间应该怎么协作；
- 哪些东西通过云盘。

需求变更：喜欢云盘会牵动架构

讲者说因为自己很喜欢云盘，就会问：是不是引入云盘要做一些**架构修改**？他希望**邮件和资料等都和云盘保持同步**。讲者把这件事称为**需求的更改**。他现在不知道它的架构做得好不好，于是让 Agent 帮自己看一看。

好架构的判断：需求变更时改动是否摧毁性、是否背技术债

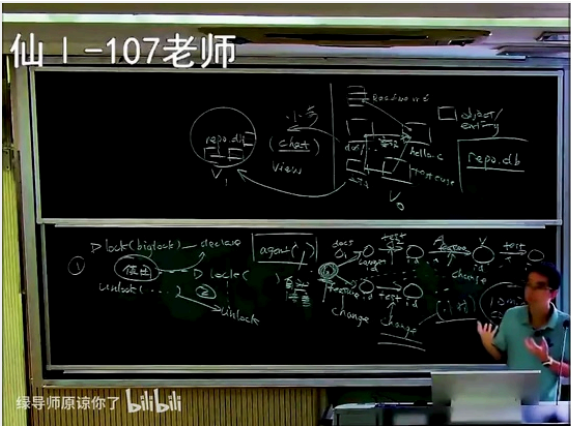
讲者的推理是：

- 当前当然有一个需求；
- 现在的实现可能满足这个需求；
- 但当需求发生变更时，相应做出的变化可能是摧毁性的，或者背上技术债，还是别的，那就知道了。

当前看到的设计：有 assistant.db，要把个人资料放到这里

讲者接着看他的设计：它有一个 assistant.db，他要把个人资料放到这里。

第 28 部分



Took 0.0s

Thinking...

你的直觉是对的，而且改动量比想象中少——因为项目从第一天就选了「Markdown 是真理，SQLite 只是索引」。但有一条硬红线必须先立起来：

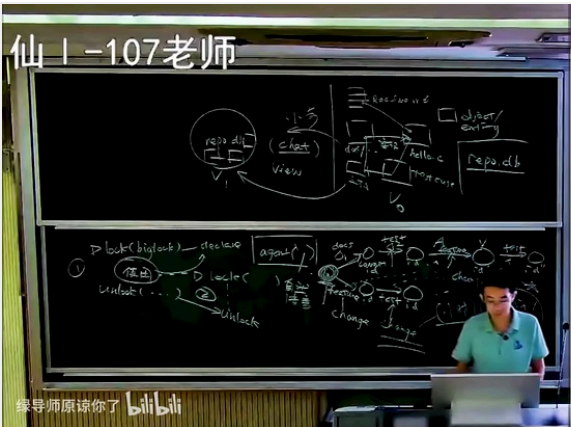
绝不把 state/assistant.db 放进云盘同步。

它是 WAL 模式的二进制流，装的是确认状态机、任务队列、IMAP 游标 (uidvalidity/last_uid)、幂等键。两台设备同时写 → 库损坏 + 重复发信 / 重复提交 ehall。正好打穿设计里最看重的幂等与闸门。

于是原则变成：云盘同步「记录」，SQLite 是每台设备从记录派生出来的本地缓存。

数据分类：哪些能上云

数据	位置	能同步吗	理由
个人资料	data/pdb/**/*.*md	✅ 双向	纯文本，人可编辑，天然云友好：含 notices/ 归档
邮件归档	需新增 mail/**	✅ 单向 (IMAP+云)	原件 .eml+附件，按 Message-ID 命名，可重放
索引	pdb_docs/chunks/fts	❌	可由 Markdown 重建，同步它没意义还易冲突
草稿/确认/任务/审计	state/assistant.db	⚠️ 只读像只读	权威状态单写者；可导出 Markdown/JSONL 供他设备看
IMAP 游标	网上	❌	设备本地，共享会导致漏拉/重复拉
凭据	secrets.json	❌	已被 redactConfig 保护，绝不上云



用 MVP 验证需求

Minimum Viable Product: 用一个最小的版本验证

“ The biggest risk in product development is building something nobody wants.”

- MVP 会给你一个**立即能使用、不完备**的版本
- 帮助你考虑未来潜在的变化、评估软件的设计

先验证什么？

- + 先选一个最值得验证的假设，再做能验证它的最小版本。(Eric Ries: MVP)

是否算技术债没有定论，但 system.db 的设计短期不炸

- 讲者先回应前面的问题：这种设计“是背上一个技术债还是别的”，他说“那就知道了”。
- 他看这个设计：有一个 **system.db**（转录原文：SISTANT点dB）；把个人资料放这里，邮件归档也放这里，索引、草稿仍然在 db。
- 讲者判断：现在看起来还不会引起很大问题，因为“绝大部分都是这样”，数据库本身确实不应该引起特别大的问题。
- 但这不是终点，因为随着项目不断进展，问题和开发方式有关。

两种开发方式：先做大拖技术债，或先做最小精巧架构

- 方式 A：一上来就做一个大的，几大步把需求都实现完，然后拖着技术债不断往后迭代，继续加功能。
- 方式 B：从最小开始，得到一个特别精巧的架构。
- 讲者举前面的例子：当你用 **git** 管理学生的提交以后，很多事情变得非常容易，自己控制起来也轻松很多。
- 由此讲者说：直到遇到性能瓶颈之前，他都不想去把它迁移到任何一种数据库。
- 逻辑链：
 1. system.db 把个人资料、邮件归档、索引、草稿放在一起，短期看问题不大。
 2. 项目继续走，常见一种做法是先做大、拖技术债、再加功能。
 3. 另一种做法是从最小精巧架构起步，像 git 管学生提交一样，很多事更容易控制。
 4. 因此不必过早迁移数据库；把迁移推迟到性能瓶颈出现。
 5. （整理者串联）这里的隐含取舍是：越早采用最小可用架构，越能把“何时迁移/重构”推到真正有性能压力时，而不是被早期大设计拖住。

早期 prompt 对项目走向影响最大

- 讲者结论：早期的 **prompt** 对整个项目的走向影响最大。
- 例子：最早直接把 **README** 给 AI，并说“我要做一个 MVP”，都会对项目造成很大的影响。
- 讲者强调，最早的这些输入会持续塑造后续方向。

下一节转向软件架构和构造

- 讲者说，下一节课、也就是周日，会进入**软件架构和构造**的内容。

小结：讲者用 system.db 与 git 管学生提交的对比说明：早期设计/prompt 决定项目是拖技术债还是保持最小精巧，迁移数据库应等到性能瓶颈出现。
